รายงานการทดลอง

เรื่อง รีเลย์เชิงผลต่างสำหรับการป้องกันหม้อแปลงไฟฟ้า 3 เฟส

(Differential Relay for Three-Phase Transformer Protection)

นางสาวจิรัชยาพร	ศิริมงคลชัยกูล	รหัสนักศึกษา 65010140
นายชวัลวิทย์	รวมมิตรสมัคร	รหัสนักศึกษา 65010199
นายโชติวุฒ	สำราญมาก	รหัสนักศึกษา 65010232
นายฌานวิทย์	แก้วใหญ่	รหัสนักศึกษา 65010236
นางสาวณัฏฐ์ชพร	สารเล็ก	รหัสนักศึกษา 65010283
นางสาวณัฐฐาพร	ดวงชัย	รหัสนักศึกษา 65010293
นายณัฐพล	สีนา	รหัสนักศึกษา 65010311
นายณัฐวัตร	ศิริคุรุรัตน์	รหัสนักศึกษา 65010328
นายธนาธิป	กาวไธสง	รหัสนักศึกษา 65010441
นายธเนศ	สุขศรี	รหัสนักศึกษา 65010448
นายพงษ์พาดา	อะโนมา	รหัสนักศึกษา 65010666
นายพชร	คุณาภรณ์	รหัสนักศึกษา 65010668
นายรชต	พงศ์สุทธิ	รหัสนักศึกษา 65010901
นายรัชตะ	เงี้ยวพ่าย	รหัสนักศึกษา 65010919
นายวิทวัช	อินแก้ว	รหัสนักศึกษา 65010992

เสนอ

รศ. ดร.อรรถพล เง่าพิทักษ์กุล

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 01026323 POWER SYSTEM PROTECTION

หลักสูตรวิศวกรรมไฟฟ้า ภาควิชาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ประจำภาคการศึกษาที่ 2 ปีการศึกษา 2568

สารบัญ

บทที่ 1 บทนำ 1

1.1 ความเป็นมาและความสำคัญของปัญหา 1

1.2 วัตถุประสงค์ของโครงงาน 1

1.3 ขอบเขตของโครงงาน 1

1.4 ประโยชน์ที่คาดว่าจะได้รับ 2

บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง 3

2.1 Inter-Turn Fault ในหม้อแปลง 3 เฟส และข้อจำกัดของ Overcurrent Relay 3

2.2 หลักการ Differential Protection และ Percentage Bias Characteristic 3

2.3 Dual-Slope Characteristic และ Knee Point 4

2.4 หม้อแปลงกระแส (CT): Class 0.5 และ Burden 5

2.5 ไมโครคอนโทรลเลอร์ STM32F411 และการวัด RMS ด้วย ADC 5

2.6 งานวิจัยที่เกี่ยวข้อง 5

บทที่ 3 วิธีการดำเนินงานและออกแบบระบบ 6

3.1 ภาพรวมกระบวนการดำเนินงาน 6

3.2 การรวบรวมและจัดระเบียบข้อมูล 7

3.3 การคัดกรองสัญญาณ 7

3.3.1 การตรวจสอบโครงสร้างไฟล์ 7

3.3.2 การคัดกรองด้วย I1 Ratio 7

3.3.3 การหา Transition Row ด้วย Sliding RMS Window 8

3.4 การตรวจสอบสัญญาณด้วย MATLAB (script_1) 8

3.5 การวิเคราะห์ Bias Characteristic (script_2) 8

3.5.1 Scatter Plot 8

3.5.2 Grid Search 9

3.6 การทดสอบอัลกอริทึม (script_3) 9

3.6.1 หลักการจำลอง 9

3.6.2 ผลการทดสอบ 10

3.7 การวิเคราะห์ Worst-Case Noise (script_4) 10

3.7.1 แบบจำลอง Noise Budget 10

3.7.2 ผลการวิเคราะห์ 11

3.8 การออกแบบฮาร์ดแวร์ 11

3.8.1 รายการอุปกรณ์ 11

3.8.2 วงจร Signal Conditioning 12
3.8.3 การต่อวงจรบน Protoboard 12
3.8.4 การ Calibrate CT แต่ละช่อง (GAIN_CH) 13
3.9 การพัฒนาซอฟต์แวร์ 14
3.9.1 สถาปัตยกรรมซอฟต์แวร์ 14
3.9.2 Boot Sequence และ Calibration 14
3.9.3 Dual-Slope Protection Characteristic 14
3.9.4 การ Upload Firmware 15
บทที่ 4 ผลการทดลองและวิเคราะห์ผล 16
4.1 ผลการวิเคราะห์ Bias Characteristic และ Grid Search 16
4.2 ผลการจำลองอัลกอริทึม (Detection Rate / False Positive) 17
4.3 ผลการวิเคราะห์ Worst-Case Noise 17
4.4 ผลการทดสอบบน Hardware จริง 18
4.4.1 กรณีทดสอบที่ 1: แทป 762-381 (50%) เฟส A แรงดัน 45 V 18
4.4.2 กรณีทดสอบที่ 2: แทป 762-457 (40%) เฟส B แรงดัน 55 V 19
4.4.3 กรณีทดสอบที่ 3: แทป 762-457 (40%) เฟส C แรงดัน 55 V 20
4.4.4 สรุปผลการทดสอบบน Hardware จริง 20
บทที่ 5 สรุปผลและข้อเสนอแนะ 21
5.1 สรุปผลการดำเนินงาน 21
5.2 ปัญหาและข้อจำกัดที่พบ 21
5.3 ข้อเสนอแนะสำหรับการพัฒนาต่อ 21
เอกสารอ้างอิง 23
ภาคผนวก 24
รายชื่อสัญญาณที่ผ่านการคัดกรอง 24
สคริปต์ script_1_plot_selected_signal.m 29
สคริปต์ script_2_bias_characteristic.m 32
สคริปต์ script_3_simulate_relay.m 40
สคริปต์ script_4_simulate_relay_worstcase_noise.m 46
Firmware STM32_Differential_Relay_V9.ino 54

บทที่ 1
บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

หม้อแปลงไฟฟ้าเป็นอุปกรณ์ที่มีความสำคัญอย่างยิ่งในระบบส่งและจำหน่ายพลังงานไฟฟ้า หากเกิดความผิดปกติหรือความเสียหายขึ้น ย่อมส่งผลกระทบโดยตรงต่อความมั่นคงของระบบไฟฟ้าและผู้ใช้งาน ความผิดปกติที่อันตรายที่สุดประการหนึ่งสำหรับหม้อแปลงคือ การลัดวงจรภายในโดยเฉพาะอย่างยิ่งการลัดวงจรข้ามรอบขดลวด (Inter-turn Fault) ซึ่งอาจเกิดจากการเสื่อมสภาพของฉนวน

ปัญหาสำคัญของการลัดวงจรข้ามรอบคือในระยะเริ่มต้นกระแสโหลดทางฝั่งทุติยภูมิอาจไม่ได้เพิ่มสูงขึ้นอย่างมีนัยสำคัญ ทำให้รีเลย์ป้องกันกระแสเกิน (Overcurrent Relay) แบบมาตรฐานไม่สามารถตรวจจับความผิดปกติได้ทันท่วงที ในขณะเดียวกัน ภายในตัวหม้อแปลงจะเกิดกระแสไหลวน (Circulating Current) ปริมาณมหาศาลบริเวณขดลวดที่ลัดวงจร ซึ่งก่อให้เกิดความร้อนสะสมอย่างรวดเร็วและอาจนำไปสู่การระเบิดหรือเพลิงไหม้ได้

ด้วยเหตุนี้รีเลย์ป้องกันผลต่าง (Differential Relay) จึงถูกนำมาใช้เพื่อแก้ปัญหานี้โดยเฉพาะ เนื่องจากทำงานบนหลักการเปรียบเทียบกระแสไฟฟ้าที่ไหลเข้าและออกจากตัวอุปกรณ์ หากมีกระแสรั่วไหลหรือลัดวงจรภายใน ค่าผลต่างของกระแสจะไม่เท่ากับศูนย์และรีเลย์จะสั่งตัดวงจรทันที อย่างไรก็ตาม รีเลย์ผลต่างในระดับอุตสาหกรรมมักมีราคาสูงและเป็นระบบปิด

ดังนั้น โครงงานนี้จึงมีแนวคิดที่จะศึกษา ออกแบบ และสร้างแบบจำลองการทำงานของรีเลย์ป้องกันผลต่าง โดยประยุกต์ใช้ไมโครคอนโทรลเลอร์ประสิทธิภาพสูง STM32 ร่วมกับหม้อแปลงกระแส (Current Transformer : CT) ร่วมกับวงจรต่างๆ วัดกระแสไฟฟ้าและพัฒนาระบบประมวลผลสัญญาณดิจิทัลที่สามารถคำนวณสมการ Percentage Bias ได้อย่างแม่นยำ พร้อมทั้งทดสอบประสิทธิภาพการทำงานด้วยชุดข้อมูลจริงจากห้องปฏิบัติการ เพื่อยืนยันความสามารถในการตรวจจับการลัดวงจรภายในได้อย่างถูกต้องและรวดเร็ว

1.2 วัตถุประสงค์ของโครงงาน

- เพื่อออกแบบและสร้างแบบจำลองการทำงานของรีเลย์ผลต่าง
- เพื่อพัฒนาโปรแกรมบนไมโครคอนโทรลเลอร์ STM32 สำหรับประมวลผลสัญญาณไฟฟ้า
- เพื่อทดสอบประสิทธิภาพการทำงานร่วมกับข้อมูลจริง จากห้องปฏิบัติการ

1.3 ขอบเขตของโครงงาน

โครงงานนี้มีขอบเขตการดำเนินงานครอบคลุม 4 ด้านหลัก ดังนี้

ด้านฮาร์ดแวร์ ต้นแบบสร้างบนไมโครคอนโทรลเลอร์ STM32F411CEU6 (Black Pill) ทำงานที่ความถี่ 100 MHz โดยใช้หม้อแปลงกระแสชนิด Solid-Core MSQ-0.66 อัตราส่วนแปลง 30A/5A Class 0.5 จำนวน 6 ตัวสำหรับวัดกระแสทั้งฝั่งปฐมภูมิและทุติยภูมิของหม้อแปลง 3 เฟส วงจร Signal Conditioning แต่ละช่องประกอบด้วยตัวต้านทาน Burden 0.1 Ω วงจรแบ่งแรงดัน DC Bias 1.65 V และวงจรป้องกัน

แรงดันเกินด้วย Zener 3.3 V การแสดงผลใช้จอ OLED 1.3 นิ้ว SH1106 และแจ้งเตือนด้วย Buzzer MH-FMD

ด้านเฟิร์มแวร์และอัลกอริทึม เฟิร์มแวร์เวอร์ชัน 9 พัฒนาด้วย Arduino IDE บน STM32duino ใช้โครงสร้าง Non-blocking timer แยก Sampling loop (1 kHz), RMS/Protection loop (40 ms) และ OLED update loop (250 ms) ออกจากกัน อัลกอริทึมป้องกันใช้ Dual-Slope Percentage Bias Characteristic โดยคำนวณ EFFECTIVE_RATIO อัตโนมัติจากสัญญาณขณะ Boot Calibration ทำให้รองรับหม้อแปลงทุกประเภท (Yy/Yd/Dy/Dd) โดยไม่ต้อง Hardcode ค่าใดๆ ล่วงหน้า

ด้านข้อมูลและการวิเคราะห์ ใช้ข้อมูลสัญญาณกระแสที่บันทึกด้วย Power Quality Analyzer Hioki PQ3198 อัตราการสุ่มตัวอย่าง 20 kHz จากการทดลองจำลอง Inter-Turn Fault (ITF) บนหม้อแปลง 3 เฟสจริงในห้องปฏิบัติการ ครอบคลุมกลุ่มการทดลอง G01–G15 รวมทั้งสิ้นประมาณ 100 ไฟล์ ตัวแปรที่ศึกษาได้แก่ ระดับแรงดัน ขนาดโหลด เปอร์เซ็นต์แทปที่เกิดการลัดวงจร และเฟสที่เกิดความผิดปกติ (A/B/C) การวิเคราะห์และปรับพารามิเตอร์ใช้ MATLAB ผ่านชุดสคริปต์ 4 ชุด ได้แก่ การพล็อตสัญญาณ การวิเคราะห์ Bias Characteristic, Grid Search และการจำลองอัลกอริทึมทั้งแบบปกติและแบบ Worst-Case Noise

ด้านข้อจำกัด โครงงานนี้ทดสอบเฉพาะกรณี Inter-Turn Fault ที่เกิดภายในหม้อแปลงเท่านั้น ไม่ครอบคลุมความผิดปกติประเภทอื่น เช่น Phase-to-Phase Fault หรือ Earth Fault ต้นแบบที่สร้างขึ้นเป็นระบบ Single-Phase ต่อ 1 ชุด CT ต่อเฟส ไม่ได้รวมระบบ Trip Breaker จริง และไม่ได้ผ่านการรับรองมาตรฐานอุตสาหกรรม

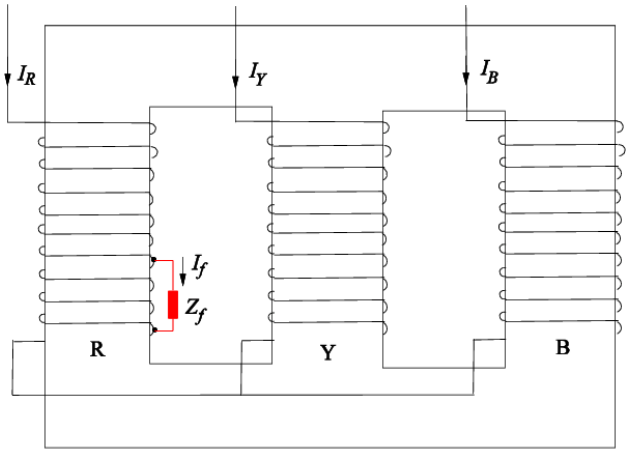
1.4 ประโยชน์ที่คาดว่าจะได้รับ

โครงงานนี้ทำให้ได้ต้นแบบดิจิทัลรีเลย์เชิงผลต่าง (Digital Differential Relay) ที่มีต้นทุนต่ำและสามารถทำงานได้จริงบนไมโครคอนโทรลเลอร์ STM32 พร้อมทั้งเป็นการยืนยันประสิทธิภาพของอัลกอริทึม Dual-Slope Percentage Bias ว่าสามารถตรวจจับการลัดวงจรข้ามรอบ (Inter-Turn Fault) ได้อย่างแม่นยำ 100% โดยไม่มีการทริปผิดพลาด นอกจากนี้ยังช่วยให้ผู้จัดทำได้บูรณาการความรู้เชิงปฏิบัติ เกิดความเข้าใจอย่างลึกซึ้งเกี่ยวกับพฤติกรรมของระบบป้องกันหม้อแปลงไฟฟ้า การออกแบบวงจรฮาร์ดแวร์ และการพัฒนาระบบประมวลผลสัญญาณดิจิทัลแบบเรียลไทม์

บทที่ 2
ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

2.1 Inter-Turn Fault ในหม้อแปลง 3 เฟส และข้อจำกัดของ Overcurrent Relay

หม้อแปลงไฟฟ้าเป็นอุปกรณ์สำคัญในระบบส่งและจำหน่ายพลังงาน ความเสียหายที่อันตรายที่สุดประเภทหนึ่งคือ Inter-Turn Fault (ITF) ซึ่งเกิดจากการเสื่อมสภาพของฉนวนระหว่างรอบขดลวดภายในหม้อแปลง ทำให้เกิดการลัดวงจรภายในบางส่วนของขดลวดดังรูปที่ 2.1 ในระยะเริ่มต้น กระแสทางฝั่งทุติยภูมิอาจไม่เพิ่มสูงขึ้นอย่างมีนัยสำคัญ เนื่องจากจำนวนรอบที่เกี่ยวข้องอาจมีน้อย แต่ภายในบริเวณที่ลัดวงจรจะมีกระแสวนขนาดใหญ่ไหลอยู่ ก่อให้เกิดความร้อนสะสมอย่างรวดเร็วและอาจนำไปสู่การระเบิดหรือเพลิงไหม้ได้ [1]



รูปที่ 2.1 วงจรการเกิด ITF ภายในหม้อแปลงสามเฟส

Overcurrent Relay แบบมาตรฐานไม่สามารถตรวจจับ ITF ในระยะเริ่มต้นได้ เนื่องจากกระแสด้านทุติยภูมิที่วัดได้ยังอยู่ในระดับปกติ การศึกษาของ Hettiwatte et al. [2] แสดงให้เห็นว่า ITF ที่เกี่ยวข้องกับรอบขดลวดน้อยกว่า 10% สามารถเกิดขึ้นโดยไม่ทำให้กระแสโหลดเกินค่า pickup ของ overcurrent relay ทั่วไป จึงจำเป็นต้องใช้หลักการป้องกันที่แตกต่างออกไป

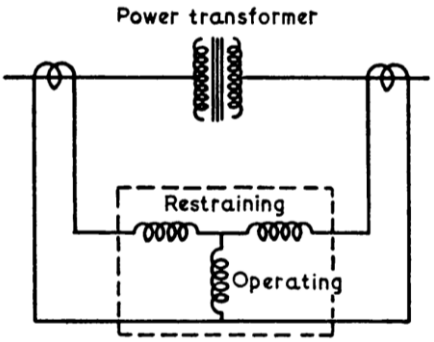
2.2 หลักการ Differential Protection และ Percentage Bias Characteristic

Differential Relay ทำงานบนหลักการเปรียบเทียบกระแสที่ไหลเข้าและออกจากอุปกรณ์ที่ต้องการป้องกัน หากผลต่างของกระแส (I_diff) มีค่าเกินเกณฑ์ที่กำหนด แสดงว่ามีกระแสรั่วไหลภายในและรีเลย์จะสั่งตัดวงจร สำหรับหม้อแปลงไฟฟ้า กระแสที่เข้าและออกจะถูกวัดผ่าน Current Transformer (CT) ทั้งสองด้าน โดยนิยามตัวแปรหลักดังนี้ [3]

$\mathrm{I_diff} = |\mathrm{I_pri_comp} - \mathrm{I_sec}|$ และ $\mathrm{I_bias} = (\mathrm{I_pri_comp} + \mathrm{I_sec}) / 2$

โดยที่ I_pri_comp คือกระแสปฐมภูมิที่ถูกปรับ Scale ด้วย EFFECTIVE_RATIO เพื่อให้สามารถเปรียบเทียบกับกระแสทุติยภูมิได้โดยตรง

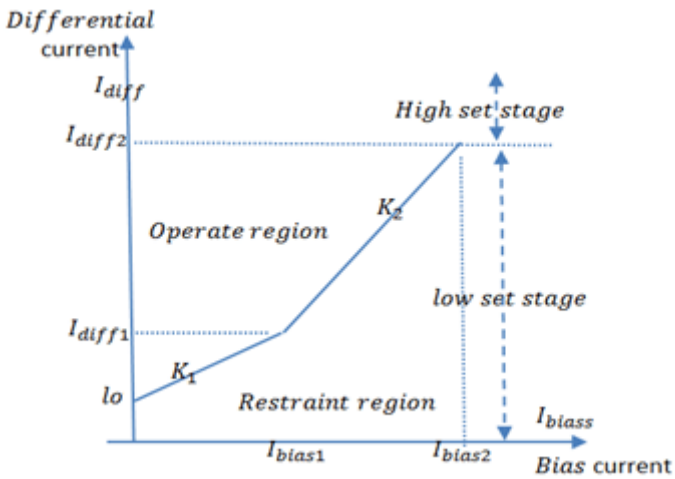
อย่างไรก็ตาม ในทางปฏิบัติ CT และหม้อแปลงจริงย่อมมีความคลาดเคลื่อนเล็กน้อยอยู่เสมอ หากใช้เพียงค่า I_diff คงที่เป็นเกณฑ์ตัดสิน จะเกิด False Trip ในสภาวะโหลดสูง แนวทางที่ใช้อย่างแพร่หลายในอุตสาหกรรมคือ Percentage Bias Characteristic ซึ่งกำหนดให้ Threshold ของ I_diff แปรผันตาม I_bias ทำให้ระบบยืดหยุ่นมากขึ้นในช่วงกระแสสูง ซึ่งประกอบด้วยขดลวด Restraining และ Operating ดังรูป 2.2 [4]



รูปที่ 2.2 Percentage Differential Relay

2.3 Dual-Slope Characteristic และ Knee Point

มาตรฐาน IEC 60255-111 [5] กำหนดลักษณะ Dual-Slope สำหรับรีเลย์ป้องกันเชิงผลต่าง โดยแบ่งลักษณะเส้น Characteristic ออกเป็น 2 ช่วงตาม Knee Point (I_k) ดังรูปที่ 2.3



รูปที่ 2.3 Differential Relay Dual-Slope Characteristic

ช่วงที่ 1 $(\mathrm{I_bias} < \mathrm{I_k})$: $\mathrm{Threshold} = \mathrm{Slope1} \times \mathrm{I_bias} + \mathrm{Pickup}$ ใช้สำหรับช่วงกระแสต่ำถึงปานกลาง เพื่อให้มีความไวในการตรวจจับ ITF ที่กระแสต่ำ

ช่วงที่ 2 $(\mathrm{I_bias} \geq \mathrm{I_k})$: $\mathrm{Threshold} = \mathrm{Slope2} \times \mathrm{I_bias} - \mathrm{OFFSET2}$ ใช้ค่า Slope ที่สูงกว่าในช่วงกระแสสูง เพื่อรองรับ Through-Fault ที่อาจทำให้ CT เกิด Saturation และเพิ่มค่า I_diff ขึ้นโดยไม่ใช่ความผิดปกติจริง โดยที่ OFFSET2 คำนวณจากเงื่อนไข Continuity ที่จุด Knee: OFFSET2 = (Slope2 – Slope1) × I_k – Pickup

2.4 หม้อแปลงกระแส (CT): Class 0.5 และ Burden

Current Transformer (CT) เป็นอุปกรณ์ที่แปลงกระแส AC ขนาดใหญ่ให้อยู่ในระดับที่เหมาะสม สำหรับการวัด Class ของ CT หมายถึงระดับความแม่นยำ โดย Class 0.5 ตามมาตรฐาน IEC 61869-2 [6] อนุญาตให้มี Ratio Error ไม่เกิน ±0.5% และ Phase Displacement ไม่เกิน 30 นาทีเชิงมุม (arcmin) ที่กระแสพิกัด

Burden คือค่าอิมพีแดนซ์โหลดที่ต่อกับขด Secondary ของ CT มีผลโดยตรงต่อการเลือก R_burden โครงงานนี้ใช้ CT ชนิด MSQ-0.66 อัตราส่วน 30A/5A โดยคำนวณ R_burden ได้จาก Rated Burden = 2.5 VA ดังนั้น $\mathrm{R_max} = \mathrm{VA} / \mathrm{I_rated}^2$ = 2.5 / 25 = 0.1 Ω ซึ่งให้แรงดัน $\mathrm{V_rms} = 5\mathrm{A} \times 0.1\Omega = 0.5\ \mathrm{V}$ ที่กระแสพิกัด เหมาะสมกับช่วง ADC 0–3.3 V ของ STM32

2.5 ไมโครคอนโทรลเลอร์ STM32F411 และการวัด RMS ด้วย ADC

STM32F411CEU6 เป็นไมโครคอนโทรลเลอร์ตระกูล ARM Cortex-M4 ทำงานที่ความถี่สูงสุด 100 MHz มี ADC 12-bit ความละเอียด 4096 ระดับ และรองรับ Direct Memory Access (DMA) ทำให้เหมาะสมกับงาน Real-time signal acquisition [7] การคำนวณ RMS ใช้วิธี Accumulate Sum-of-Squares แบบ Non-blocking โดยสะสมค่า ADC^2 ทุก 1 ms แล้วคำนวณ RMS พร้อมกับการตัดสิน Trip ในทุก 40 ms ซึ่งสอดคล้องกับการสะสม 2 รอบของความถี่ 50 Hz

2.6 งานวิจัยที่เกี่ยวข้อง

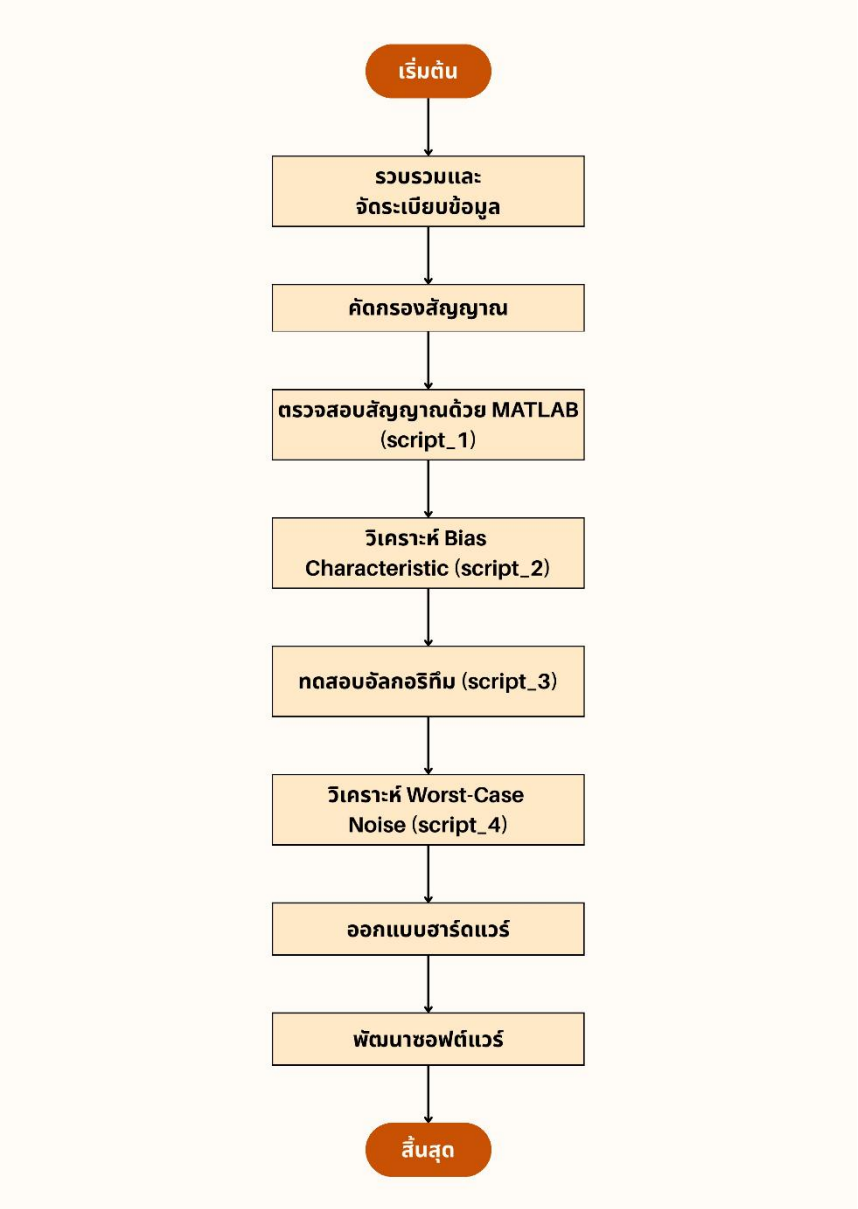
Balaga et al. [8] นำเสนอการใช้ Percentage Bias Differential Relay ร่วมกับ Harmonic Restraint เพื่อแยก Inrush Current ออกจาก Internal Fault โดยแสดงให้เห็นว่าค่า Slope ที่เหมาะสมขึ้นอยู่กับพิสัยของ CT Error และ Load Level Faiz และ Lotfi-Fard [9] เสนอวิธีการตรวจจับ ITF โดยอาศัยการวิเคราะห์ Negative Sequence Current ซึ่งมีความไวสูงกว่าการวัดกระแสโดยตรง แต่ต้องการฮาร์ดแวร์ที่ซับซ้อนกว่า งานวิจัยของ Tripathy et al. [10] ได้ทำการศึกษาการใช้ Artificial Neural Network ในการแยกแยะประเภทความผิดปกติในหม้อแปลง ซึ่งแสดงให้เห็นถึงข้อจำกัดของวิธีการแบบ Rule-based ในกรณีที่มีสัญญาณรบกวน

โครงงานนี้เลือกใช้แนวทาง Percentage Bias Dual-Slope เนื่องจากมีหลักการที่ตรงไปตรงมา ทดสอบได้บน Hardware จริงที่ราคาประหยัด และสอดคล้องกับมาตรฐาน IEC 60255-111

บทที่ 3
วิธีการดำเนินงานและออกแบบระบบ

3.1 ภาพรวมกระบวนการดำเนินงาน

การดำเนินงานโครงการแบ่งออกเป็น 3 ส่วนหลัก ได้แก่ การวิเคราะห์และประมวลผลข้อมูลสัญญาณ การออกแบบวงจรฮาร์ดแวร์ และการพัฒนาซอฟต์แวร์บนไมโครคอนโทรลเลอร์ STM32F411 โดยมีลำดับขั้นตอนดังแสดงในรูปที่ 3.1



รูปที่ 3.1 ภาพรวมกระบวนการดำเนินงาน

3.2 การรวบรวมและจัดระเบียบข้อมูล

ข้อมูลสัญญาณที่ใช้ในโครงการได้จากการทดลองวัดสัญญาณกระแสด้วย Power Quality Analyzer Hioki PQ3198 ที่อัตราการสุ่มตัวอย่าง 20 kHz บันทึกในรูป CSV แต่ละไฟล์มีจำนวน 5,600 แถว ครอบคลุมสภาวะการทำงานปกติ (Healthy) ต่อเนื่องไปจนถึงสภาวะ Inter-Turn Fault (ITF)

ไฟล์สัญญาณนำมาเรียบเรียงใหม่และจัดเก็บแยกตามกลุ่มการทดลอง (G01–G15) ในโฟลเดอร์ Signal_G*/ โดยตั้งชื่อตามประเภทการทดลองดังตารางที่ 3.1

ชื่อไฟล์	ประเภทการทดลอง	ตัวแปรที่เปลี่ยน
2.2.x_*V.csv	ITF + เปลี่ยนแรงดัน	ระดับแรงดัน (V)
2.3.x_*W.csv	ITF + เปลี่ยนโหลด	ขนาดโหลด (W)
2.4.x_*.csv	ITF + เปลี่ยนแทป	เปอร์เซ็นต์แทปที่ short
2.5.x_*.csv	ITF + เปลี่ยน Phase	Phase A / B / C ที่เกิด ITF

ตารางที่ 3.1 ประเภทไฟล์สัญญาณที่ใช้ในการทดลอง

3.3 การคัดกรองสัญญาณ

เนื่องจากข้อมูลที่บันทึกมาบางส่วนอยู่ในช่วงที่เกิด ITF แล้วและมีการเปลี่ยนโหลด (กระแสฝั่ง Primary สูงแต่แทปไม่เปลี่ยน ในขณะที่กระแสฝั่ง Secondary ลดลง) จึงต้องคัดกรองเฉพาะไฟล์ที่บันทึกสัญญาณก่อน-หลังเกิด ITF อย่างสมบูรณ์ โดยมีขั้นตอนดังนี้

3.3.1 การตรวจสอบโครงสร้างไฟล์

ตรวจสอบว่า EventNo มีเพียงค่าเดียวตลอดทั้งไฟล์ ซึ่งยืนยันว่าไม่มี state label ฝังอยู่ในข้อมูล จึงต้องตรวจสอบ transition จากสัญญาณโดยตรง

3.3.2 การคัดกรองด้วย I1 Ratio

คำนวณค่า RMS ของกระแส I1 (Primary) ในช่วง 100 แถวแรก (ช่วง Healthy) และ 100 แถวท้าย (ช่วง Fault) จากนั้นคำนวณอัตราส่วน

$$\text{ratio} = \text{RMS(I1_end)} \ / \ \text{RMS(I1_start)}$$

เกณฑ์การคัดกรอง: ratio ≥ 2 ถือว่า OK (มี transition ชัดเจน) และ ratio < 2 ถือว่า SKIP

ผลการคัดกรองพบว่าไฟล์ที่ถูก SKIP ส่วนใหญ่เป็นไฟล์ประเภท 2.5.2_B ซึ่ง ITF เกิดที่ Phase B แต่ I1 วัด Phase อื่น จึงไม่เห็น transition ใน I1 ซึ่งถูกต้องตามหลักการ ไม่ใช่ข้อมูลเสีย โดยมีไฟล์ที่ถูก SKIP ทั้งสิ้น 16 ไฟล์ และมีการดึงไฟล์ สำหรับ G15_Dy/2.4.1_10 ที่ยืนยันฟอลต์จาก waveform จริง

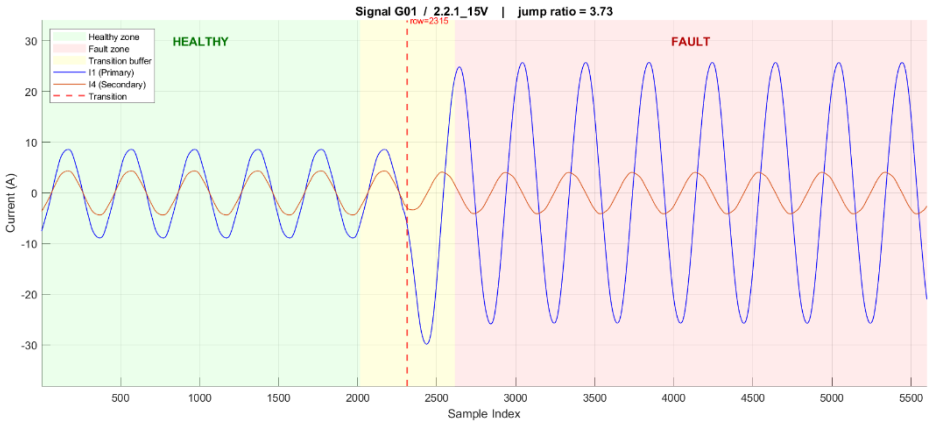
3.3.3 การหา Transition Row ด้วย Sliding RMS Window

หลังคัดกรองด้วย Ratio แล้ว ใช้ Sliding RMS Window ขนาด 200 samples เลื่อนตลอดไฟล์ คำนวณอัตราส่วน RMS ก่อน/หลัง window แถวที่ให้ Ratio สูงสุดคือ transition point โดยกำหนด buffer 300 samples รอบ transition เพื่อหลีกเลี่ยง transient

ไฟล์ที่ผ่านการคัดกรองถูกคัดลอกไปยังโฟลเดอร์ Selected_Signals/ รวมทั้งสิ้นประมาณ 100 ไฟล์

3.4 การตรวจสอบสัญญาณด้วย MATLAB (script_1)

สคริปต์ script_1_plot_selected_signal.m ทำการพลอตสัญญาณ I1 (Primary, สีน้ำเงิน) และ I4 (Secondary, สีส้ม) พร้อม shaded region ที่แสดง Healthy / Transition buffer / Fault ให้เห็นภาพรวมของทุกไฟล์ เพื่อตรวจสอบด้วยสายตาว่าไฟล์ที่เลือกมามีสัญญาณก่อน-หลัง ITF จริงหรือไม่



รูปที่ 3.2 ตัวอย่างผลการพลอตสัญญาณด้วย script_1 แสดง Healthy zone (เขียว) Transition buffer (เหลือง) และ Fault zone (แดง)

หลังตรวจสอบ waveform ทั้งหมดแล้ว พบว่าสัญญาณที่ผ่านการคัดกรองมีรูปแบบสอดคล้องกับที่คาดไว้ทุกไฟล์

3.5 การวิเคราะห์ Bias Characteristic (script_2)

เพื่อกำหนดพารามิเตอร์ของ Dual-Slope Characteristic ที่เหมาะสม จึงพัฒนา script_2_bias_characteristic.m ซึ่งทำงาน 2 ส่วนควบคู่กัน

3.5.1 Scatter Plot

คำนวณ I_diff และ I_bias ของทุก 40 ms window ในทุกไฟล์ แล้ว plot บน I_bias–I_diff plane โดยแยกสีระหว่าง Healthy windows (น้ำเงิน) และ Fault windows (แดง) พร้อม overlay characteristic curve ตามพารามิเตอร์ที่ตั้งไว้ใน Section 1 ของสคริปต์ทำให้สามารถปรับค่า SLOPE1, SLOPE2, PICKUP, KNEE_POINT ด้วยการสังเกตภาพได้ทันที

3.5.2 Grid Search

นอกจาก scatter plot แล้ว script_2 ยังทำ Grid Search หาชุดพารามิเตอร์ (PICKUP, SLOPE1, SLOPE2, KNEE_POINT) ที่ทำให้ False Positive = 0% และ Detection Rate = 100% บน dataset ทั้งหมดพร้อมกัน โดยคำนวณ Continuity Condition สำหรับ OFFSET2 อัตโนมัติ:

$$\text{OFFSET2} = (\text{SLOPE2 - SLOPE1}) \times \text{KNEE_POINT - PICKUP}$$

ผลลัพธ์ถูก export เป็นไฟล์ grid_search_results.csv (ชุดพารามิเตอร์ทั้งหมดที่ผ่านเกณฑ์ เรียงตาม composite margin) และ grid_search_best.txt (พารามิเตอร์ที่ดีที่สุด พร้อม copy ไปใส่ใน firmware ได้ทันที)

3.6 การทดสอบอัลกอริทึม (script_3)

หลังได้พารามิเตอร์ที่เหมาะสมจาก script_2 แล้ว ทำการทดสอบด้วย script_3_simulate_relay.m ซึ่งจำลอง firmware v9 บน recorded signal ทุกไฟล์ใน Selected_Signals/ โดยใช้วิธี Block Window (non-overlapping 40 ms window) เนื่องจากพัฒนาได้ง่ายกว่า Sliding Window และสอดคล้องกับ firmware จริงมากกว่า

3.6.1 หลักการจำลอง

ขั้นตอน	รายละเอียด
หา Transition Row	Sliding RMS Window เหมือนขั้นตอนคัดกรอง
แบ่ง Zone	Healthy = ก่อน transition−300, Fault = หลัง transition+300
Calibrate ER	$\text{ER} = \text{RMS}(\text{I_pri_healthy_last10cycle}) / \text{RMS}(\text{I_sec_healthy_last10cycle})$
คำนวณต่อ Window	$\text{I_pri_comp} = \text{I_pri_rms} / \text{ER}, \text{I_diff} = \lvert\text{I_pri_comp} - \text{I_sec_rms}\rvert, \text{I_bias} = (\text{I_pri_comp} + \text{I_sec_rms})/2$
ตัดสิน	เปรียบ I_diff กับ Dual-Slope threshold
เกณฑ์ PASS	False Positive = 0% และ Detection = 100%

ตารางที่ 3.2 ขั้นตอนการจำลองอัลกอริทึมใน script_3

3.6.2 ผลการทดสอบ

อัลกอริทึม Block Window สามารถตรวจจับ ITF ได้ 100% ในทุกไฟล์ที่ป้อนเข้า และ False Positive = 0% ในส่วน Healthy ผลการจำลองถูกบันทึกใน simulation_results.csv (สรุปรายไฟล์) และ simulation_log.csv (time series ทุก window)

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	File	WindowIdx	TimeSec	Zone	I_pri_rms	I_sec_rms	I_pri_comp	I_diff	I_bias	Threshold	Trip			
2	2.2.1_15V.csv	Signal_G01	1	0.01	H	6.2266	3.0944	3.0937	0.0007	3.0941	0.7594	0		
3	2.2.1_15V.csv	Signal_G01	2	0.05	H	6.2265	3.0943	3.0936	0.0007	3.094	0.7594	0		
4	2.2.1_15V.csv	Signal_G01	3	0.1308	F	18.1473	2.8612	9.0165	6.1554	5.9388	1.4194	1		
5	2.2.1_15V.csv	Signal_G01	4	0.1708	F	18.2366	2.8711	9.0609	6.1899	5.966	1.433	1		
6	2.2.1_15V.csv	Signal_G01	5	0.2107	F	18.2407	2.8779	9.0629	6.1851	5.9704	1.4352	1		
7	2.2.2_30V.csv	Signal_G01	1	0.0638	F	31.684	3.6601	15.7345	12.0744	9.6973	3.2986	1		
8	2.2.2_30V.csv	Signal_G01	2	0.1038	F	31.625	3.6891	15.7051	12.016	9.6971	3.2986	1		
9	2.2.2_30V.csv	Signal_G01	3	0.1438	F	31.5931	3.7163	15.6893	11.973	9.7028	3.3014	1		
10	2.2.2_30V.csv	Signal_G01	4	0.1838	F	31.6115	3.7418	15.6984	11.9566	9.7201	3.3101	1		
11	2.2.2_30V.csv	Signal_G01	5	0.2238	F	31.6266	3.7634	15.7059	11.9425	9.7347	3.3173	1		
12	2.3.1_1200W.csv	Signal_G01	1	0.01	H	4.1547	2.024	2.024	0	2.024	0.6524	0		
13	2.3.1_1200W.csv	Signal_G01	2	0.05	H	4.1548	2.024	2.024	0	2.024	0.6524	0		
14	2.3.1_1200W.csv	Signal_G01	3	0.0901	H	4.1553	2.0242	2.0243	0.0001	2.0242	0.6524	0		
15	2.3.1_1200W.csv	Signal_G01	4	0.13	H	4.1544	2.0239	2.0238	0.0001	2.0239	0.6524	0		
16	2.3.1_1200W.csv	Signal_G01	5	0.2379	F	48.5959	1.5582	23.6735	22.1154	12.6159	4.7579	1		
17	2.3.2_2400W.csv	Signal_G01	1	0.01	H	8.1596	4.0262	4.0239	0.0024	4.025	0.8525	0		
18	2.3.2_2400W.csv	Signal_G01	2	0.05	H	8.1606	4.0271	4.0244	0.0027	4.0257	0.8526	0		
19	2.3.2_2400W.csv	Signal_G01	3	0.0901	H	8.1605	4.0269	4.0243	0.0025	4.0256	0.8526	0		
20	2.3.2_2400W.csv	Signal_G01	4	0.13	H	8.1615	4.0273	4.0248	0.0025	4.0261	0.8526	0		
21	2.3.2_2400W.csv	Signal_G01	5	0.22	F	48.0901	3.1077	23.7155	20.6078	13.4116	5.1558	1		

รูปที่ 3.3 ตัวอย่างผลการจำลองอัลกอริทึม แสดง I_diff ช่วง Healthy และ Fault เทียบกับ threshold

3.7 การวิเคราะห์ Worst-Case Noise (script_4)

เพื่อยืนยันความน่าเชื่อถือของระบบเมื่อนำไปใช้กับ Hardware จริง จึงทำการวิเคราะห์ผลกระทบของ noise จากวงจร ADC และ CT ด้วย script_4_simulate_relay_worstcase_noise.m

3.7.1 แบบจำลอง Noise Budget

แหล่ง noise หลักประกอบด้วย 2 ส่วน:

- ADC offset noise: worst-case 2 LSB × CT_ratio × R_burden ≈ 0.0097 A ต่อช่อง
- CT Class 0.5 error: ratio error ±0.5% และ phase error 30 arcmin ⟶ combined fraction ≈ 1.30%

สมการ worst-case noise floor

$$\mathrm{I_diff_noise(I_bias)} \approx 0.0194 + 0.026 \times \mathrm{I_bias}\ \ [\mathrm{A}]$$

วิธีการ inject noise: $\mathrm{I_pri_noisy} = \mathrm{I_pri} \times (1 + \mathrm{CT_error}) + \mathrm{ADC_offset}$ และ $\mathrm{I_sec_noisy} = \mathrm{I_sec} \times (1 - \mathrm{CT_error}) - \mathrm{ADC_offset}$ เพื่อให้ noise ส่งผลในทิศทางที่เพิ่ม I_diff มากที่สุด

3.7.2 ผลการวิเคราะห์

ผลพบว่าแม้เพิ่ม worst-case noise เข้าไปในสัญญาณแล้ว อัลกอริทึมยังคงให้ผล PASS ทุกไฟล์ (False Positive = 0%, Detection = 100%) แสดงว่า noise margin ที่ออกแบบไว้เพียงพอสำหรับการใช้งานจริง

3.8 การออกแบบฮาร์ดแวร์

3.8.1 รายการอุปกรณ์

ตารางที่ 3.3 แสดงรายการอุปกรณ์ที่ใช้สร้างต้นแบบ Digital Differential Relay พร้อมทั้งจำนวนและจุดประสงค์ในการใช้งาน

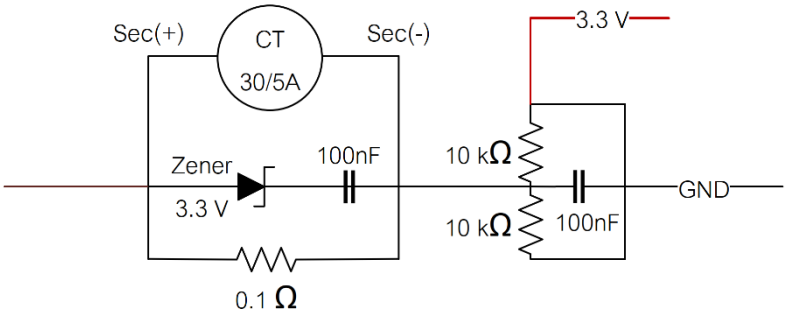
รายการ	Specification	จำนวน	หมายเหตุ
STM32F411CEU6 Black Pill	ARM Cortex-M4 @ 100 MHz	1	MCU หลัก
ST-LINK V2	USB Programmer	1	Upload โค้ด
CT MSQ-0.66	30A/5A, Class 0.5	6	วัดกระแส 3 เฟส × 2 ด้าน
OLED 1.3" SH1106	I2C 0x3C, 128×64 px	1	แสดงผล I_diff, I_bias
Buzzer MH-FMD	Active 3.3-5V, PB0	1	ส่งเสียงเตือนเมื่อ Fault
R 0R1 (0.1Ω) 2W 5%		6	R_burden
R 10kΩ Metal Film 1/4W 1%		12	Voltage divider DC bias 1.65V
C 10µF 16V Electrolytic		6	Bypass capacitor
C 100nF Ceramic (104)		6	Filter high-frequency noise
Zener 3.3V 0.5W		6	Over-voltage protection ที่ ADC

ตารางที่ 3.3 รายการอุปกรณ์ฮาร์ดแวร์

3.8.2 วงจร Signal Conditioning

วงจร Signal Conditioning มีหน้าที่แปลงกระแส AC จาก CT ให้เป็นแรงดัน 0–3.3V สำหรับ ADC ของ STM32 โดยประกอบด้วย 3 ส่วนหลักได้แก่

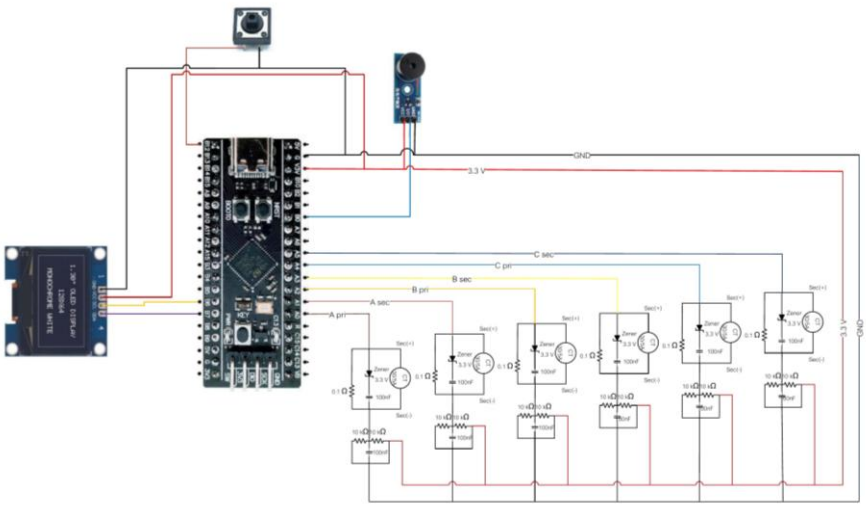
- R_burden (0R1, 0.1Ω 2W 5% ตัวเดี่ยว): แปลง I_CT เป็น V_AC ที่ $V_rms = 5A \times 0.1\Omega = 0.5V\ rms$ ที่กระแสพิกัด
- DC Bias 1.65V: Voltage divider จาก R 10kΩ สองตัวต่อระหว่าง 3.3V และ GND ทำให้คลื่น AC ถูกยก offset ขึ้นมาที่ศูนย์กลาง 1.65V ไม่ถูกตัด
- Protection: Zener 3.3V คร่อม ADC pin ป้องกันแรงดันเกิน, C 100nF กรอง high-frequency noise



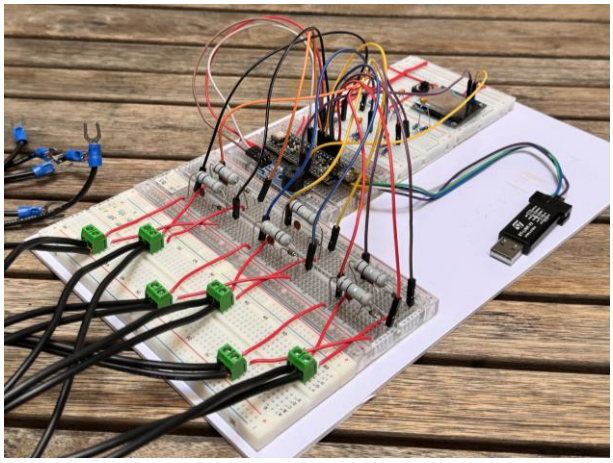
รูปที่ 3.4 วงจร Signal Conditioning สำหรับ 1 ช่อง CT (ทำซ้ำ × 6 ช่อง)

3.8.3 การต่อวงจรบน Protoboard

หลังออกแบบวงจรแล้วทำการจัดซื้ออุปกรณ์และต่อวงจรบน Protoboard โดยต่อวงจร Signal Conditioning ทั้ง 6 ช่อง (Primary A/B/C และ Secondary A/B/C) พร้อมกับเชื่อมต่อกับ STM32 Black Pill ผ่าน Jumper Wire และสายแข็งเบอร์ AWG 22



รูปที่ 3.5 Digital Differential Relay Schematic Diagram



รูปที่ 3.6 วงจรต้นแบบ Digital Differential Relay บน Protoboard

3.8.4 การ Calibrate CT แต่ละช่อง (GAIN_CH)

หลังต่อวงจรบน Protoboard เสร็จสมบูรณ์แล้ว ทำการ Calibrate CT ทั้ง 6 ช่องโดยป้อนกระแสที่ 60–80% ของพิกัด CT (ประมาณ 18–24 A) วัดค่าจริงด้วย HIOKI AC CLAMP POWER METER CM3286 และเปรียบเทียบกับค่าที่ STM32 อ่านได้ผ่าน ADC จากนั้นคำนวณ Gain Correction ของแต่ละช่องดังนี้

$$\mathrm{GAIN} = \mathrm{I_clamp} \,/\, \mathrm{I_stm}$$

| Primary | | |

ค่า GAIN ที่สูงกว่า 1.0 สะท้อนให้เห็นว่า CT แต่ละตัวมี Ratio Error อยู่ในช่วง 10–15% ซึ่งสูงกว่าสเปก Class 0.5 (±0.5%) อย่างมีนัยสำคัญ สาเหตุหลักมาจากการเสียแรงดันบน R_burden และ Jumper Wire บน Protoboard ค่า GAIN_CH นี้ทำหน้าที่ชดเชยความคลาดเคลื่อนดังกล่าว และจะถูกรวมเข้าไปใน EFFECTIVE_RATIO โดยอัตโนมัติในขั้นตอน Boot Calibration อีกชั้นหนึ่ง

3.9 การพัฒนาซอฟต์แวร์

3.9.1 สถาปัตยกรรมซอฟต์แวร์

Firmware เขียนด้วย Arduino IDE (STM32duino) บน STM32F411 โดยใช้ Non-blocking timer เพื่อให้งานหลายอย่างทำงานพร้อมกันโดยไม่มี delay() ใน loop:

- Sampling loop (1 kHz): สุ่มตัวอย่าง ADC ทุก 6 ช่องทุก 1 ms สะสมค่า sum of squares
- RMS + Protection (40 ms): คำนวณ RMS และตัดสิน shouldTrip() ทุก 40 ms
- OLED Update (250 ms): แสดงผลบนหน้าจอทุก 250 ms
- Button Debounce: ตรวจจับการกดปุ่ม PB12 แบบ non-blocking เพื่อสลับ phase display

3.9.2 Boot Sequence และ Calibration

เมื่อเปิดเครื่อง ระบบทำ Calibration อัตโนมัติ 2 ขั้นตอน:

1. DC Offset Calibration (~3 วินาที): วัดค่า DC offset ของ ADC แต่ละช่องโดยไม่มีโหลด เพื่อนำไปหักออกตอนวัดจริง
2. Effective Ratio Calibration (~10 วินาที): วัด I_pri และ I_sec ขณะมีโหลดปกติ แล้วคำนวณ EFFECTIVE_RATIO = I_pri_avg / I_sec_avg ค่านี้รวม turn ratio + connection type (√3 factor) + CT mismatch ทุกอย่างไว้ในตัวเดียวโดยอัตโนมัติ

การใช้ EFFECTIVE_RATIO เป็นจุดเด่นของ firmware v9 ทำให้รองรับหม้อแปลงทุกประเภท (Yy/Yd/Dy/Dd) โดยไม่ต้อง hardcode ค่า turn ratio หรือ delta factor ล่วงหน้า

3.9.3 Dual-Slope Protection Characteristic

การตัดสิน trip ใช้ฟังก์ชัน shouldTrip(I_diff, I_bias) ซึ่งเปรียบ I_diff กับ threshold ของ Dual-Slope characteristic:

ถ้า $\text{I_bias} < \text{KNEE_POINT}: \ \text{threshold} = \text{SLOPE1} \times \text{I_bias} + \text{PICKUP}$

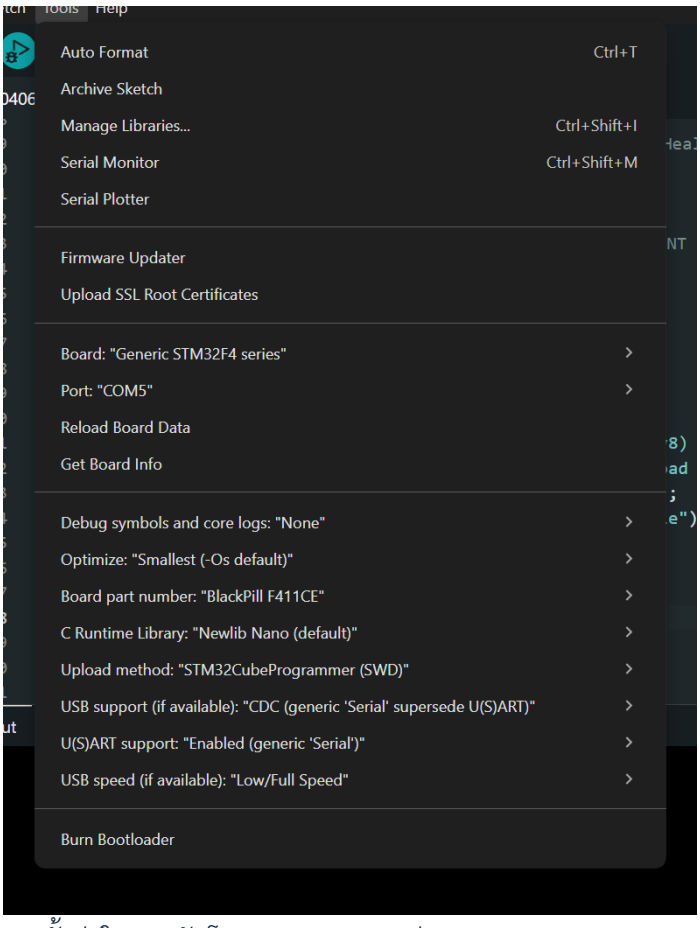
ถ้า $\text{I_bias} \geq \text{KNEE_POINT}: \ \text{threshold} = \text{SLOPE2} \times \text{I_bias} - \text{OFFSET2}$

Trip เมื่อ: I_diff > threshold

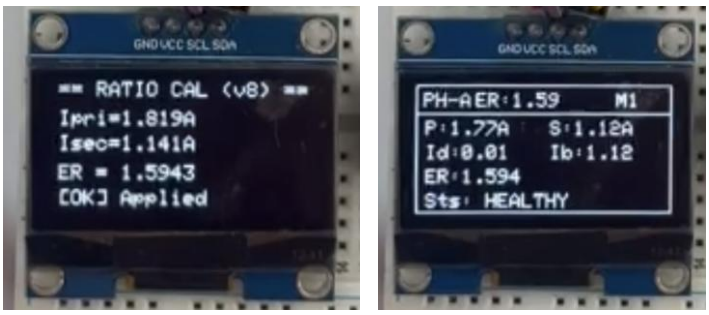
Slope 1 ให้ความไวในการตรวจจับ fault ที่กระแสต่ำ และ Slope 2 ที่ชันกว่ามีหน้าที่ลด False Trip จาก Through-fault ในช่วงกระแสสูง

3.9.4 การ Upload Firmware

ทำการ upload firmware STM32_Differential_Relay_V9.ino ไปยัง STM32F411 ผ่าน ST-LINK V2 โดยใช้ Arduino IDE 2.x กับ Board Support Package "STM32 MCU based boards" ตั้งค่า Upload Method เป็น STM32CubeProgrammer (SWD) และการตั้งค่าอื่นๆ ตามรูปที่ 3.7



รูปที่ 3.7 การตั้งค่าในการอัพโหลด Firmware ผ่าน ST-LINK V2 และ Arduino IDE



รูปที่ 3.8 ตัวอย่างหน้าจอ OLED แสดงผลหลัง Boot Calibration สำเร็จ แสดง I_diff, I_bias, ER และสถานะของเฟสที่เลือกอยู่

บทที่ 4
ผลการทดลองและวิเคราะห์ผล

4.1 ผลการวิเคราะห์ Bias Characteristic และ Grid Search

จากการนำข้อมูลสัญญาณที่ผ่านการคัดกรองจากทุกกลุ่ม (G01–G15) ประมาณ 100 ไฟล์มาคำนวณ I_diff และ I_bias ในทุก Window ขนาด 40 ms พบว่า Healthy Windows กระจายตัวอยู่ในโซนต่ำของระนาบ I_bias–I_diff โดย I_diff มีค่าอยู่ในช่วง 0.0–0.3 A ในขณะที่ Fault Windows มีค่า I_diff สูงกว่าอย่างชัดเจน บางกรณีสูงถึง 2–4 A ขึ้นอยู่กับเปอร์เซ็นต์แทปที่เกิดการลัดวงจร Scatter Plot จาก script_2 แสดงให้เห็นว่ามีช่องว่างระหว่างสองกลุ่มที่ชัดเจน ซึ่งยืนยันความเป็นไปได้ในการแยกแยะ Healthy จาก Fault ด้วย Dual-Slope Characteristic

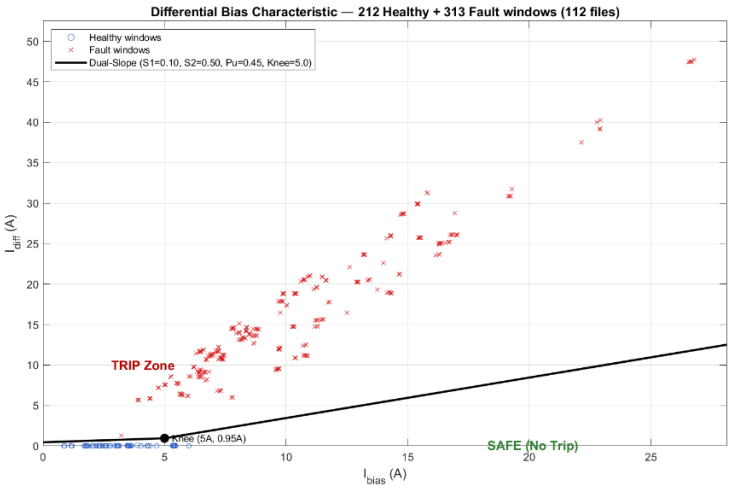
การทำ Grid Search โดยค้นหาชุดพารามิเตอร์ (Pickup, Slope1, Slope2, Knee Point) ที่ให้ False Positive = 0% และ Detection Rate = 100% บน Dataset ทั้งหมดพร้อมกัน โดยมีจำนวนการทดสอบรวมกว่า 3,400 ชุด พบว่ามีชุดพารามิเตอร์ที่ผ่านเกณฑ์ทั้งสองข้อจำนวนหนึ่ง พารามิเตอร์ที่ดีที่สุดเมื่อพิจารณาจาก Composite Margin (ค่าต่ำสุดของ Margin ฝั่ง Healthy และ Fault) ที่ถูกเลือกใช้ในเฟิร์มแวร์ คือ Slope1 = 0.10, Pickup = 0.45 A, Knee Point = 5.0 A, Slope2 = 0.50 และ OFFSET2 = 1.55 A

```
grid_search_best.txt
=== Grid Search Best Result ===
FP=0%% AND Detection=100%% passing sets: 1920

Best (max composite margin = 0.5245 A):
  PICKUP     = 0.450 A
  SLOPE1     = 0.100
  SLOPE2     = 0.500
  KNEE_POINT = 5.0 A
  OFFSET2    = 1.5500 A  (calculated: (S2-S1)*Knee - Pickup)
  Margin_H   = 0.5333 A  (healthy safety margin)
  Margin_F   = 0.5245 A  (fault detection margin)

--- Copy to firmware (.ino) ---
const float SLOPE1     = 0.100f;
const float PICKUP     = 0.450f;
const float KNEE_POINT = 5.0f;
const float SLOPE2     = 0.500f;
const float OFFSET2    = 1.5500f;
```

รูปที่ 4.1 ผลลัพธ์การทำ Grid Search



รูปที่ 4.2 Bias Scatter Plot โดยใช้ชุดพารามิเตอร์ที่ดีที่สุดที่ได้จากการทำ Grid Search

4.2 ผลการจำลองอัลกอริทึม (Detection Rate / False Positive)

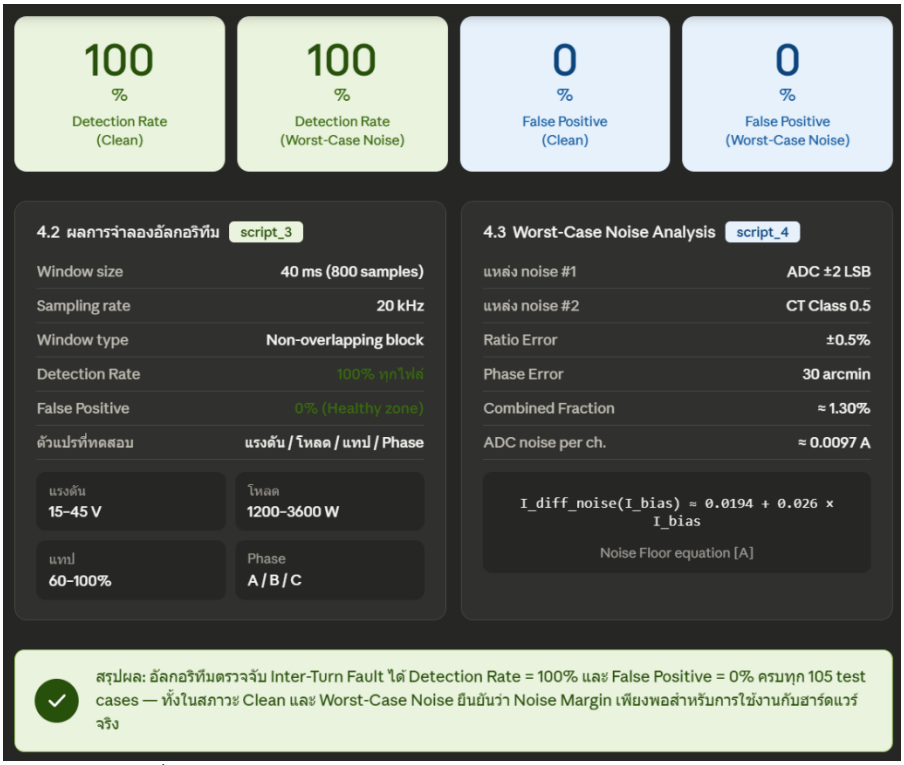
การทดสอบด้วย script_3 ซึ่งจำลองการทำงานของเฟิร์มแวร์ v9 บนสัญญาณที่บันทึกไว้ทุกไฟล์ โดยใช้ Non-overlapping Block Window ขนาด 40 ms (800 samples @ 20 kHz) และคำนวณ EFFECTIVE_RATIO จากช่วง Healthy ก่อนเกิด Fault พบว่าอัลกอริทึมสามารถตรวจจับ Inter-Turn Fault ได้ Detection Rate = 100% ในทุกไฟล์ที่ผ่านการคัดกรอง พร้อมกับ False Positive = 0% ในช่วง Healthy ผลนี้ครอบคลุมทุกตัวแปร ได้แก่ การเปลี่ยนแรงดัน การเปลี่ยนโหลด การเปลี่ยนเปอร์เซ็นต์แทป ตั้งแต่ 100% ลงถึง 60% และทุก Phase (A/B/C)

ค่า I_diff เฉลี่ยในช่วง Fault สูงกว่า Threshold อย่างมีนัยสำคัญในทุกกรณี ยืนยันว่าระยะห่างระหว่าง Operating Point กับ Characteristic Curve มีเพียงพอสำหรับการตรวจจับที่เชื่อถือได้

4.3 ผลการวิเคราะห์ Worst-Case Noise

การจำลอง Worst-Case Noise ด้วย script_4 ทำโดยการ Inject ความคลาดเคลื่อนสูงสุดจาก 2 แหล่งในทิศทางที่ทำให้ I_diff เพิ่มขึ้นมากที่สุด ได้แก่ ความคลาดเคลื่อนของ ADC (2 LSB × CT_ratio × R_burden ≈ 0.0097 A ต่อช่อง) และความคลาดเคลื่อนของ CT Class 0.5 (Ratio Error ±0.5% และ Phase Error 30 arcmin รวมกันได้ Combined Fraction ≈ 1.30%) สมการ Noise Floor ที่ได้คือ I_diff_noise(I_bias) ≈ 0.0194 + 0.026 × I_bias [A]

ผลการจำลองพบว่าแม้ใส่ Worst-Case Noise เข้าไปแล้ว อัลกอริทึมยังคงให้ผล False Positive = 0% และ Detection Rate = 100% ในทุกไฟล์ ยืนยันว่า Noise Margin ที่ออกแบบไว้เพียงพอสำหรับการใช้งานกับฮาร์ดแวร์จริง



รูปที่ 4.3 Dashboard สรุปผลการจำลองอัลกอริทึม

4.4 ผลการทดสอบบน Hardware จริง

หลังจากผ่านการตรวจสอบด้วยการจำลองอัลกอริทึมและการวิเคราะห์ Worst-Case Noise บนข้อมูลที่บันทึกไว้แล้ว คณะผู้จัดทำได้ทำการทดสอบระบบรีเลย์ต้นแบบบน Hardware จริงในห้องปฏิบัติการ โดยใช้หม้อแปลง 3 เฟสที่มีฝั่ง Primary ต่อแบบ Delta (Δ) และฝั่ง Secondary ต่อแบบ Star (Y) จำลองสภาวะการลัดวงจรระหว่างรอบขดลวด (Inter-Turn Short Circuit) ด้วยการลัดวงจรแทปที่กำหนด

การทดสอบครอบคลุม 3 กรณี ซึ่งแตกต่างกันในด้านตำแหน่งแทปที่เกิดความผิดปกติ เปอร์เซ็นต์ความต่างขนาดแรงดัน เฟสที่ทดสอบ และระดับแรงดันป้อนเข้า โดยคงขนาดโหลดไว้ที่ 3,600 W ตลอดการทดสอบ ในแต่ละกรณี ระบบและโหลดถูกเปิดใช้งานและรอให้กระบวนการ Boot Calibration เสร็จสมบูรณ์ก่อน จึงตรวจสอบสถานะที่หน้าจอ OLED ในสภาวะปกติ จากนั้นจึงจำลองสภาวะฟอลต์และบันทึกการตอบสนองของระบบ

4.4.1 กรณีทดสอบที่ 1: แทป 762-381 (50%) เฟส A แรงดัน 45 V

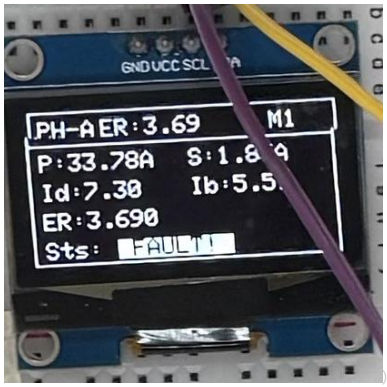
กรณีทดสอบแรกใช้การลัดวงจรระหว่างแทป 762 และ 381 ซึ่งสอดคล้องกับความต่างขนาดแรงดันระหว่างแทปขดลวดที่ 50% โดยทดสอบที่เฟส A แรงดันป้อนเข้า 45 V และโหลด 3,600 W

แทปขดลวด	% ความต่างขนาดแรงดันระหว่างแทปขดลวด	ขนาดแรงดันป้อนเข้า (V)	เฟส	ขนาดโหลด (W)	การทำงานของอุปกรณ์ทดสอบ		
					สภาวะปกติก่อนเกิดฟอลต์	เกิดฟอลต์	สภาวะปกติหลังเกิดฟอลต์
762-381	50%	45	A	3,600	✓	✓	✓

ตารางที่ 4.1 ผลการทดสอบกรณีที่ 1: แทป 762-381 เฟส A แรงดัน 45 V

จากตารางที่ 4.1 ระบบตอบสนองได้อย่างถูกต้องครบทั้ง 3 สภาวะ กล่าวคือ ในสภาวะปกติก่อนเกิดฟอลต์ หน้าจอ OLED แสดงค่า I_diff และ I_bias ในระดับปกติโดยไม่มีการทริป เมื่อเกิดฟอลต์ระบบตรวจจับและแจ้งเตือนทันทีและเมื่อยุติสภาวะฟอลต์แล้วระบบกลับสู่สภาวะปกติโดยไม่มีการทริปผิดพลาด

รูปที่ 4.4 หน้าจอ OLED กรณีที่ 1 ในสภาวะปกติก่อนเกิดฟอลต์



รูปที่ 4.5 หน้าจอ OLED กรณีที่ 1 ระหว่างเกิดฟอลต์

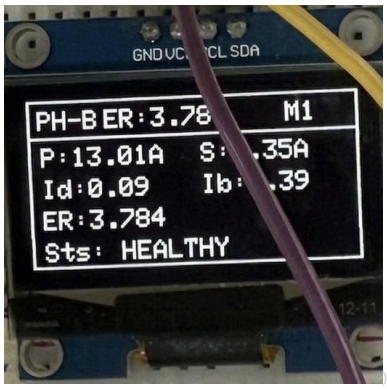
4.4.2 กรณีทดสอบที่ 2: แทป 762-457 (40%) เฟส B แรงดัน 55 V

กรณีทดสอบที่สองใช้การลัดวงจรระหว่างแทป 762 และ 457 ซึ่งสอดคล้องกับความต่างขนาดแรงดันระหว่างแทปขดลวดที่ 40% กรณีนี้เปลี่ยนเฟสทดสอบเป็น B และเพิ่มแรงดันป้อนเข้าเป็น 55 V เพื่อตรวจสอบความสามารถของระบบในการตรวจจับฟอลต์ที่เฟสอื่นและระดับแรงดันที่ต่างออกไป

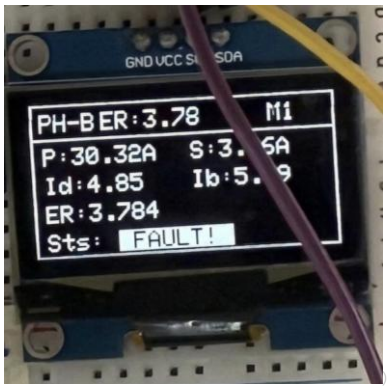
แทป ขดลวด	% ความต่างขนาดแรงดันระหว่างแทปขดลวด	ขนาดแรงดันป้อนเข้า (V)	เฟส	ขนาดโหลด (W)	การทำงานของอุปกรณ์ทดสอบ		
					สภาวะปกติก่อนเกิดฟอลต์	เกิดฟอลต์	สภาวะปกติหลังเกิดฟอลต์
762-457	40%	55	B	3,600	✓	✓	✓

ตารางที่ 4.2 ผลการทดสอบกรณีที่ 2: แทป 762-457 เฟส B แรงดัน 55 V

จากตารางที่ 4.2 ระบบสามารถตรวจจับ Inter-Turn Fault ที่เฟส B ได้อย่างถูกต้อง แม้ว่าเปอร์เซ็นต์ความต่างแรงดันจะลดลงเหลือ 40% เมื่อเทียบกับกรณีแรก ยืนยันว่าอัลกอริทึม Dual-Slope Percentage Bias มีความไวเพียงพอต่อการตรวจจับฟอลต์ขนาดเล็กลง โดยไม่มีการทริปผิดพลาดในสภาวะปกติทั้งก่อนและหลังเกิดฟอลต์



รูปที่ 4.6 หน้าจอ OLED กรณีที่ 2 ในสภาวะปกติก่อนเกิดฟอลต์



รูปที่ 4.7 หน้าจอ OLED กรณีที่ 2 ระหว่างเกิดฟอลต์

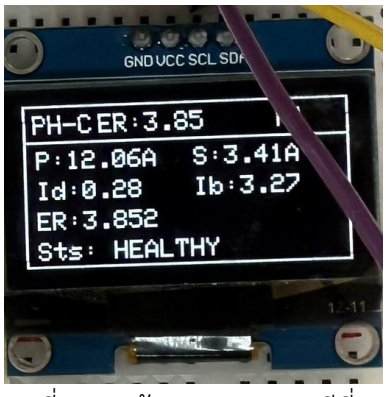
4.4.3 กรณีทดสอบที่ 3: แทป 762-457 (40%) เฟส C แรงดัน 55 V

กรณีทดสอบที่สามใช้การลัดวงจรระหว่างแทป 762 และ 457 ปรับแรงดันป้อนเข้าเป็น 55 V เช่นเดียวกับกรณีที่สอง แต่เปลี่ยนเฟสทดสอบเป็น C เพื่อให้ครอบคลุมการทดสอบครบทั้ง 3 เฟสของหม้อแปลง ทั้งนี้ระดับแทปเดิมทำให้ความต่างขนาดแรงดันอยู่ที่ 40% ในเงื่อนไขนี้

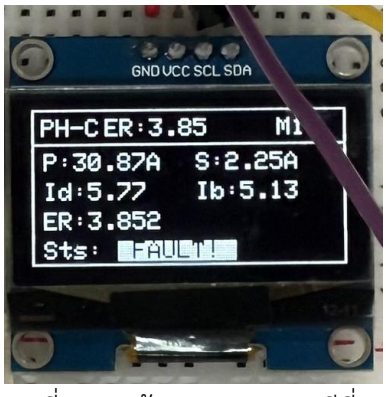
แทป ขดลวด	% ความต่างขนาดแรงดันระหว่างแทปขดลวด	ขนาดแรงดันป้อนเข้า (V)	เฟส	ขนาดโหลด (W)	การทำงานของอุปกรณ์ทดสอบ		
					สภาวะปกติก่อนเกิดฟอลต์	เกิดฟอลต์	สภาวะปกติหลังเกิดฟอลต์
762-457	40%	55	C	3,600	✓	✓	✓

ตารางที่ 4.3 ผลการทดสอบกรณีที่ 3: แทป 762-457 เฟส C แรงดัน 55 V

จากตารางที่ 4.3 ระบบตอบสนองได้ถูกต้องในทุกสภาวะเช่นเดียวกับสองกรณีก่อนหน้า การที่ระบบสามารถตรวจจับฟอลต์ได้ครบทั้งเฟส A B และ C ยืนยันว่าการออกแบบ Firmware v9 และ EFFECTIVE_RATIO Calibration สามารถรองรับการทำงานได้อย่างสมมาตรทั้งสามเฟสโดยไม่ต้องปรับพารามิเตอร์เพิ่มเติม



รูปที่ 4.8 หน้าจอ OLED กรณีที่ 3 ในสภาวะปกติก่อนเกิดฟอลต์



รูปที่ 4.9 หน้าจอ OLED กรณีที่ 3 ระหว่างเกิดฟอลต์

4.4.4 สรุปผลการทดสอบบน Hardware จริง

จากการทดสอบทั้ง 3 กรณีบนหม้อแปลง 3 เฟส Δ-Y สรุปได้ว่าระบบรีเลย์ต้นแบบผ่านการทดสอบครบทุกสถานการณ์ โดยไม่มีการทริปผิดพลาด (False Positive) ในสภาวะปกติ และตรวจจับฟอลต์ได้อย่างถูกต้องในทุกกรณี ครอบคลุมทั้ง 3 เฟส (A B C) แรงดันป้อนเข้า 45 V และ 55 V และความต่างขนาดแรงดัน 40% และ 50% ผลการทดสอบบน Hardware จริงมีความสอดคล้องกับผลการจำลองด้วย MATLAB ในหัวข้อ 4.2 และ 4.3 อย่างสมบูรณ์ ยืนยันประสิทธิภาพของอัลกอริทึม Dual-Slope Percentage Bias ที่ได้นำไปใช้จริงในเฟิร์มแวร์ v9

บทที่ 5
สรุปผลและข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

โครงงานนี้ประสบความสำเร็จในการออกแบบ สร้าง และทดสอบต้นแบบ Digital Differential Relay สำหรับป้องกันหม้อแปลงไฟฟ้า 3 เฟสจากการลัดวงจร Inter-Turn Fault โดยใช้ไมโครคอนโทรลเลอร์ STM32F411 ร่วมกับ Current Transformer MSQ-0.66 และวงจร Signal Conditioning ที่ออกแบบขึ้นเอง

เฟิร์มแวร์ v9 ที่พัฒนาขึ้นสามารถทำ Effective Ratio Calibration อัตโนมัติเมื่อ Boot ทำให้รองรับหม้อแปลงทุกประเภทโดยไม่ต้อง Hardcode ค่าใดๆ อัลกอริทึม Dual-Slope Percentage Bias ที่ได้รับการปรับพารามิเตอร์ผ่าน MATLAB Grid Search สามารถตรวจจับ Inter-Turn Fault ได้ Detection Rate = 100% และ False Positive = 0% บน Dataset ทั้งหมดประมาณ 100 ไฟล์ ครอบคลุมตัวแปรหลากหลายทั้งในการจำลองด้วย MATLAB และการทดสอบบน Hardware จริงในห้องปฏิบัติการ

การวิเคราะห์ Worst-Case Noise ยืนยันว่าระบบมี Margin เพียงพอแม้ในกรณีที่ CT และ ADC มีความคลาดเคลื่อนสูงสุดตามสเปก แสดงถึงความแข็งแกร่งของการออกแบบ

5.2 ปัญหาและข้อจำกัดที่พบ

ปัญหาสำคัญที่พบระหว่างการพัฒนาได้แก่ ค่า OFFSET2 ในเฟิร์มแวร์จะต้องถูกคำนวณใหม่ทุกครั้งที่มีการเปลี่ยนแปลงค่า Slope1 หรือ Pickup เนื่องจากค่านี้ขึ้นอยู่กับเงื่อนไข Continuity ที่จุด Knee ซึ่งหากไม่ได้ซิงโครไนซ์กันจะทำให้ Characteristic Curve ไม่ต่อเนื่อง นอกจากนี้ในช่วงต้นของการพัฒนาพบบัก CT_RATIO Double-Multiplication ที่ทำให้ Scale ผิดพลาดถึง 100 เท่า รวมถึงปัญหา Hardcoded DC Offset และ Blocking Debounce ที่รบกวนการสุ่มตัวอย่าง

ข้อจำกัดของโครงงานคือ ต้นแบบนี้ทดสอบเฉพาะ Inter-Turn Fault เท่านั้น ไม่ครอบคลุมความผิดปกติประเภทอื่น และไม่มีระบบ Trip Breaker จริง นอกจากนี้ความแม่นยำของการวัดยังมีผลกระทบจากการเสียกำลังบนสายสัญญาณ ซึ่งถูกแก้ด้วยการ Calibration แทนการแก้ไขในระดับฮาร์ดแวร์

5.3 ข้อเสนอแนะสำหรับการพัฒนาต่อ

การพัฒนาต่อในอนาคตควรพิจารณาในสามแนวทางหลักดังนี้

1) เพิ่ม Harmonic Restraint สำหรับกรอง Inrush Current ซึ่งเป็นแหล่งของ False Trip ที่พบบ่อยในการใช้งานจริง โดยมาตรฐาน IEC 60255-111 กำหนดให้ใช้ 2nd Harmonic เป็นเกณฑ์

2) ออกแบบ PCB เพื่อลด Parasitic Resistance บนสายสัญญาณ และใช้ Op-Amp Buffer เพื่อลด Gain Error

3) การขยายการทดสอบให้ครอบคลุม Fault ประเภทอื่น เช่น Phase-to-Phase Fault และ Earth Fault รวมถึงการทดสอบการทำงานของ Trip Output ร่วมกับ Circuit Breaker จริง

สำหรับกรณี Inrush Current นั้น ในระบบต้นแบบนี้ไม่เกิดปัญหา เนื่องจากขั้นตอนการใช้งานกำหนดให้ผู้ใช้ต้องเปิดโหลดก่อนและรอให้กระแสเข้าสู่สภาวะคงตัวแล้ว จึงค่อยเปิดเครื่องรีเลย์เพื่อเริ่ม

กระบวนการ Boot Calibration ดังนั้นในขณะที่ระบบเริ่มวัดสัญญาณและตัดสินใจ Trip กระแส Inrush จึงสิ้นสุดไปแล้ว ทำให้ระบบไม่เผชิญกับ Inrush ในสภาวะการทำงานจริง อย่างไรก็ตาม ข้อจำกัดนี้หมายความว่าหากผู้ใช้เปิดหม้อแปลงหลังจากที่รีเลย์ทำงานอยู่แล้ว อาจเกิด False Trip ได้ ซึ่งการเพิ่ม 2nd Harmonic Restraint ตามมาตรฐาน IEC 60255-111 จะเป็นแนวทางแก้ไขที่เหมาะสมสำหรับการพัฒนาต่อในอนาคต

เอกสารอ้างอิง

[1] P. Bastard, P. Bertrand, and M. Meunier, "A transformer model for winding fault studies," *IEEE Trans. Power Del*., vol. 9, no. 2, pp. 690–699, 1994.

[2] S. N. Hettiwatte, Z. D. Wang, and P. A. Crossley, "Investigation of the movement of partial discharge signals in a power transformer," *IEE Proc.-Sci. Meas. Technol.*, vol. 152, no. 1, pp. 33–40, 2005.

[3] A. R. van C. Warrington, *Protective Relays: Their Theory and Practice*, vol. 1. London: Chapman & Hall, 1968.

[4] G. Ziegler, *Numerical Differential Protection: Principles and Applications*, 2nd ed. Erlangen: Publicis, 2012.

[5] IEC 60255-111:2016, *Measuring relays and protection equipment -- Part 111: Functional requirements for differential protection*, Geneva: IEC, 2016.

[6] IEC 61869-2:2012, *Instrument transformers – Part 2: Additional requirements for current transformers*, Geneva: IEC, 2012.

[7] STMicroelectronics, "STM32F411xC/E Reference Manual," RM0383, Rev. 3, 2019. [Online]. Available: https://www.st.com

[8] H. Balaga, N. Gupta, and D. N. Vishwakarma, "GA trained parallel hidden layered ANN based differential protection of three phase power transformer," *Int. J. Electr. Power Energy Syst.*, vol. 67, pp. 286–297, 2015.

[9] J. Faiz and S. Lotfi-Fard, "A novel wavelet-based algorithm for discrimination of internal faults from magnetizing inrush currents in power transformers," *IEEE Trans. Power Del.*, vol. 21, no. 4, pp. 1989–1996, 2006.

[10] M. Tripathy, R. P. Maheshwari, and H. K. Verma, "Advances in transformer protection: a review," *Electric Power Compon. Syst.*, vol. 33, no. 11, pp. 1203–1209, 2005.

ภาคผนวก

รายชื่อสัญญาณที่ผ่านการคัดกรอง

Note : ใช้คำสั่ง tree บน Linux (Window Subsystem Linux)

ml@3170Q:~/Protection Group Work 2-68/Selected_Signals$ tree

```
.
├── Signal_G01
│   ├── 2.2.1_15V.csv
│   ├── 2.2.2_30V.csv
│   ├── 2.2.3_45V.csv
│   ├── 2.3.1_1200W.csv
│   ├── 2.3.2_2400W.csv
│   ├── 2.3.3_3600W.csv
│   ├── 2.4.1_20.csv
│   ├── 2.4.2_50.csv
│   ├── 2.4.3_80.csv
│   └── 2.5.3_C.csv
├── Signal_G02
│   ├── 2.2.1_15V.csv
│   ├── 2.2.2_30V.csv
│   ├── 2.2.3_45V.csv
│   ├── 2.3.1_1200W.csv
│   ├── 2.3.2_2400W.csv
│   ├── 2.4.1_10.csv
│   └── 2.4.3_70.csv
├── Signal_G03
│   ├── 2.2.1_25.csv
│   ├── 2.2.2_45.csv
│   ├── 2.2.3_55.csv
│   ├── 2.3.1_1200.csv
```

```
│   ├──── 2.3.2_2400.csv
│   ├──── 2.4.1_30.csv
│   ├──── 2.4.2_50.csv
│   ├──── 2.4.3_80.csv
│   └──── 2.5.3_C.csv
├──── Signal_G04
│   ├──── 2.2.1_25.csv
│   ├──── 2.2.2_40.csv
│   ├──── 2.2.3_55.csv
│   ├──── 2.3.1_1200.csv
│   ├──── 2.3.2_2400.csv
│   ├──── 2.4.1_30.csv
│   ├──── 2.4.3_70.csv
│   └──── 2.5.3_C.csv
├──── Signal_G05
│   ├──── 2.2.1_25.csv
│   ├──── 2.2.2_35.csv
│   ├──── 2.2.3_45.csv
│   ├──── 2.3.1_1200W.csv
│   ├──── 2.3.2_2400W.csv
│   ├──── 2.4.1_10.csv
│   ├──── 2.4.2_40.csv
│   ├──── 2.4.3_70.csv
│   └──── 2.5.3_C.csv
├──── Signal_G07
│   ├──── 2.2.2_20V.csv
│   ├──── 2.2.2_35V.csv
│   ├──── 2.2.3_50V.csv
│   ├──── 2.3.1_1200W.csv
```

```
│   ├──── 2.3.2_2400W.csv
│   ├──── 2.3.3_3600W.csv
│   ├──── 2.4.1_20Percent.csv
│   ├──── 2.4.2_50Percent.csv
│   ├──── 2.4.3_80Percent.csv
│   └──── 2.5.3_C.csv
├──── Signal_G08
│   ├──── 2.2.1_15V.csv
│   ├──── 2.2.2_30V.csv
│   ├──── 2.2.3_45V.csv
│   ├──── 2.3.1_1200W.csv
│   ├──── 2.3.2_2400W.csv
│   ├──── 2.4.1_30.csv
│   ├──── 2.4.2_60.csv
│   ├──── 2.4.3_90.csv
│   └──── 2.5.3_C.csv
├──── Signal_G10
│   ├──── 2.2.1_35.csv
│   ├──── 2.2.2_45.csv
│   ├──── 2.2.3_55.csv
│   ├──── 2.3.1_1200.csv
│   ├──── 2.3.2_2400.csv
│   ├──── 2.4.1_20.csv
│   ├──── 2.4.3_80.csv
│   ├──── 2.5.1_A.csv
│   └──── 2.5.3_C.csv
├──── Signal_G11
│   ├──── 2.2.1_50.csv
│   ├──── 2.2.2_90.csv
```

```
│   ├──── 2.2.3_110.csv
│   ├──── 2.3.1_1200.csv
│   ├──── 2.3.2_2400.csv
│   ├──── 2.4.1_20.csv
│   ├──── 2.4.3_80.csv
│   └──── 2.5.3_C.csv
├──── Signal_G12
│   ├──── 2.2.1_50.csv
│   ├──── 2.2.2_70.csv
│   ├──── 2.2.3_90.csv
│   ├──── 2.3.1_1200.csv
│   ├──── 2.3.2_2400.csv
│   ├──── 2.3.3_3600.csv
│   ├──── 2.4.1_20.csv
│   └──── 2.4.3_80.csv
├──── Signal_G13
│   ├──── 2.2.1_15V.csv
│   ├──── 2.2.2_25V.csv
│   ├──── 2.2.3_45V.csv
│   ├──── 2.3.1_1200.csv
│   ├──── 2.3.2_2400.csv
│   ├──── 2.4.1_10.csv
│   ├──── 2.4.3_70.csv
│   └──── 2.5.2_B.csv
├──── Signal_G14
│   ├──── 2.2.1_25V.csv
│   ├──── 2.2.2_40V.csv
│   ├──── 2.2.3_55V.csv
│   ├──── 2.3.1_1200W.csv
```

```
│   ├── 2.3.2_2400W.csv
│   ├── 2.3.3_3600W.csv
│   ├── 2.4.1_60.csv
│   ├── 2.4.3_30.csv
│   └── 2.4.3_90.csv
└── Signal_G15_Dy
    ├── 2.2.1_15V.csv
    ├── 2.2.2_30V.csv
    ├── 2.2.3_45V.csv
    ├── 2.3.1_1200.csv
    ├── 2.3.2_2400.csv
    ├── 2.4.1_10.csv
    ├── 2.4.3_70.csv
    └── 2.5.3_C.csv
```

14 directories, 112 files

สคริปต์ script_1_plot_selected_signal.m

```
% plot_signals.m
% Plot I1 (primary) and I4 (secondary) with Healthy/Fault regions
% Output: Signal_Plots/<GroupName>_<filename>.png  (flat, no subfolders)

base_dir = 'Selected_Signals';
out_dir  = 'Signal_Plots';
if ~exist(out_dir, 'dir'), mkdir(out_dir); end

groups = dir(base_dir);
groups = groups([groups.isdir] & ~startsWith({groups.name}, '.'));

total_saved = 0;

for g = 1:length(groups)
    group_path = fullfile(base_dir, groups(g).name);
    files = dir(fullfile(group_path, '*.csv'));

    for f = 1:length(files)
        fpath = fullfile(group_path, files(f).name);
        fname = files(f).name;
        gname = groups(g).name;

        % Read CSV
        try
            data = readmatrix(fpath, 'NumHeaderLines', 1);
        catch
            fprintf('SKIP (read error): %s\n', fpath);
            continue
        end

        if size(data,1) < 400 || size(data,2) < 11
            fprintf('SKIP (too small): %s\n', fpath);
            continue
        end

        I1 = data(:, 8);    % Primary current
        I4 = data(:, 11);   % Secondary current
        N  = length(I1);
        t  = (1:N)';

        % --- Find transition row via sliding RMS window (on I1) ---
        win = 200;
        best_row   = win + 1;
        best_ratio = 0;

        for i = (win+1):(N-win)
            rms1 = sqrt(mean(I1(i-win:i-1).^2));
            rms2 = sqrt(mean(I1(i:i+win-1).^2));
            if rms1 > 0
                r = rms2 / rms1;
                if r > best_ratio
                    best_ratio = r;
                    best_row   = i;
                end
            end
        end
```

```
buf         = 300;
healthy_end = max(1, best_row - buf);
fault_start = min(N, best_row + buf);

% --- Axis limits (cover both I1 and I4) ---
all_vals = [I1; I4];
y_min = min(all_vals);
y_max = max(all_vals);
y_pad = (y_max - y_min) * 0.15;
y_lo  = y_min - y_pad;
y_hi  = y_max + y_pad;

% --- Plot ---
fig = figure('Visible', 'off', 'Position', [0 0 1400 550]);
hold on;

% Shaded regions
fill([1, healthy_end, healthy_end, 1], ...
     [y_lo, y_lo, y_hi, y_hi], ...
     [0.6 1.0 0.6], 'FaceAlpha', 0.20, 'EdgeColor', 'none');

fill([fault_start, N, N, fault_start], ...
     [y_lo, y_lo, y_hi, y_hi], ...
     [1.0 0.6 0.6], 'FaceAlpha', 0.20, 'EdgeColor', 'none');

fill([healthy_end, fault_start, fault_start, healthy_end], ...
     [y_lo, y_lo, y_hi, y_hi], ...
     [1.0 1.0 0.6], 'FaceAlpha', 0.30, 'EdgeColor', 'none');

% Signals
plot(t, I1, 'b-', 'LineWidth', 0.7);
plot(t, I4, 'Color', [0.85 0.33 0.10], 'LineWidth', 0.7);

% Transition line
xline(best_row, 'r--', 'LineWidth', 1.5);

% Region labels
y_label = y_hi - y_pad * 0.5;
if healthy_end > 200
    text(healthy_end/2, y_label, 'HEALTHY', ...
         'FontSize', 11, 'FontWeight', 'bold', 'Color', [0 0.45 0], ...
         'HorizontalAlignment', 'center');
end
if (N - fault_start) > 200
    text((fault_start + N)/2, y_label, 'FAULT', ...
         'FontSize', 11, 'FontWeight', 'bold', 'Color', [0.7 0 0], ...
         'HorizontalAlignment', 'center');
end
text(best_row, y_hi, sprintf(' row=%d', best_row), ...
     'FontSize', 8, 'Color', 'r', 'HorizontalAlignment', 'left');

hold off;

xlabel('Sample Index');
ylabel('Current (A)');
title(sprintf('%s  /  %s    |    jump ratio = %.2f', ...
      strrep(gname,'_',' '), strrep(fname,'.csv',''), best_ratio), ...
      'Interpreter', 'none', 'FontSize', 11);
legend({'Healthy zone','Fault zone','Transition buffer', ...
```

```
                'I1 (Primary)','I4 (Secondary)','Transition'}, ...
               'Location', 'northwest', 'FontSize', 8);
        grid on;
        ylim([y_lo, y_hi]);
        xlim([1, N]);

        % Save — flat folder, filename = GroupName_originalname.png
        safe_gname = strrep(gname, ' ', '_');
        out_name   = sprintf('%s_%s', safe_gname, strrep(fname, '.csv', '.png'));
        out_path   = fullfile(out_dir, out_name);
        exportgraphics(fig, out_path, 'Resolution', 150);
        close(fig);

        total_saved = total_saved + 1;
        fprintf('SAVED [%3d]: %s\n', total_saved, out_path);
    end
end

fprintf('\n=== Done. %d plots saved to %s/ ===\n', total_saved, out_dir);
```

สคริปต์ script_2_bias_characteristic.m

```
% ==========================================================================
% script_2_bias_characteristic.m
% Differential Bias Characteristic Analysis
%
% วัตถุประสงค์:
%   1. คำนวณ I_diff และ I_bias ของทุก window ทุกไฟล์ใน Selected_Signals/
%   2. Plot Scatter (Healthy vs Fault) บน I_bias-I_diff plane
%      พร้อม Dual-Slope characteristic curve ให้ปรับ tune ด้วยตา
%   3. Grid Search หาชุด PICKUP, SLOPE1, SLOPE2, KNEE_POINT
%      ที่ให้ FP=0% และ Detection=100% บน dataset ทั้งหมด
%      แล้ว report ออกมาเป็น CSV เรียงตาม margin ดีสุด
%
% Input  : Selected_Signals/**/*.csv
% Output :
%   Simulation_Output/bias_scatter.png          - scatter plot รวมทุกไฟล์
%   Simulation_Output/bias_characteristic.csv   - raw I_diff/I_bias ทุก window
%   Simulation_Output/grid_search_results.csv   - ผล grid search เรียงตาม margin
%   Simulation_Output/grid_search_best.txt      - ค่าที่ดีที่สุด (พร้อม copy ไป firmware)
%
% *** ปรับ TUNING PARAMETERS ใน Section 1 เพื่อเปลี่ยน curve บน scatter plot ***
% *** Grid Search range ปรับได้ใน Section 2 ***
% ==========================================================================

clear; clc;

% ==========================================================================
% 1.  PARAMETERS ที่ใช้วาด characteristic curve บน scatter plot
%     (เปลี่ยนค่าเหล่านี้เพื่อ tune ด้วยตา)
% ==========================================================================
FS              = 20000;   % Hz
F0              = 50;      % Hz
RMS_WIN         = round(0.040 * FS);   % 800 samples (40 ms)
SAMPLES_PER_CYCLE = FS / F0;           % 400

% Transition detection
WIN_TRANS       = 200;
BUFFER          = 300;
CAL_CYCLES      = 10;
TR_MIN_CURRENT  = 0.5;

% --- Characteristic curve parameters (แก้ตรงนี้เพื่อ tune ด้วยตา) ---
PICKUP_PLOT     = 0.45;   % A   - MinPickup (intercept ของ Slope 1 ที่ Ib=0)
SLOPE1_PLOT     = 0.2;    % -   - Slope ช่วง low current
SLOPE2_PLOT     = 0.60;   % -   - Slope ช่วง high current
KNEE_PLOT       = 5.0;    % A   - จุดเปลี่ยน slope (knee point)
% OFFSET2 คำนวณเองให้ curve ต่อกันที่ knee (continuity):
%   slope1*knee + pickup = slope2*knee - offset2
%   offset2 = (slope2 - slope1)*knee - pickup
OFFSET2_PLOT    = (SLOPE2_PLOT - SLOPE1_PLOT) * KNEE_PLOT - PICKUP_PLOT;

fprintf('=== Characteristic Curve Parameters ===\n');
fprintf('PICKUP    = %.3f A\n', PICKUP_PLOT);
fprintf('SLOPE1    = %.3f\n',   SLOPE1_PLOT);
fprintf('SLOPE2    = %.3f\n',   SLOPE2_PLOT);
fprintf('KNEE      = %.3f A\n', KNEE_PLOT);
```

```
fprintf('OFFSET2   = %.3f A  (auto-calculated for continuity)\n', OFFSET2_PLOT);
fprintf('  [Verify: at Ib=%.1f → thr1=%.3f, thr2=%.3f (should match)]\n', ...
    KNEE_PLOT, SLOPE1_PLOT*KNEE_PLOT+PICKUP_PLOT, SLOPE2_PLOT*KNEE_PLOT-
OFFSET2_PLOT);
fprintf('========================================\n\n');

% ========================================================================
% 2.  GRID SEARCH PARAMETERS  (range ของค่าที่จะทดสอบ)
% ========================================================================
gs_pickup  = 0.20 : 0.05 : 1.00;    % A    — MinPickup
gs_slope1  = 0.10 : 0.05 : 0.50;    % —    — Slope 1
gs_slope2  = 0.50 : 0.10 : 1.20;    % —    — Slope 2
gs_knee    = [5, 8, 10, 12, 15];    % A    — Knee point

fprintf('Grid Search: %d×%d×%d×%d = %d combinations\n', ...
    numel(gs_pickup), numel(gs_slope1), numel(gs_slope2), numel(gs_knee), ...
    numel(gs_pickup)*numel(gs_slope1)*numel(gs_slope2)*numel(gs_knee));
fprintf('(This may take a moment...)\n\n');

% ========================================================================
% 3.  FIND FILES
% ========================================================================
base_dir = 'Selected_Signals';
if ~isfolder(base_dir)
    error('Folder "%s" not found. Run from project root.', base_dir);
end
csv_files = dir(fullfile(base_dir, '**', '*.csv'));
n_files   = numel(csv_files);
if n_files == 0
    error('No CSV files found under %s', base_dir);
end
fprintf('Found %d CSV files.\n\n', n_files);

% ========================================================================
% 4.  OUTPUT SETUP
% ========================================================================
out_dir = 'Simulation_Output';
if ~isfolder(out_dir), mkdir(out_dir); end

% ========================================================================
% 5.  EXTRACT I_diff / I_bias FROM ALL FILES
% ========================================================================
% Pre-allocate accumulators
all_ib_h = [];   all_id_h = [];   % Healthy zone
all_ib_f = [];   all_id_f = [];   % Fault zone

fid_raw = fopen(fullfile(out_dir, 'bias_characteristic.csv'), 'w');
fprintf(fid_raw,
'File,Group,WindowIdx,Zone,I_pri_rms,I_sec_rms,ER,I_pri_comp,I_diff,I_bias\n');

fprintf('--- Pass 1: Extracting I_diff / I_bias ---\n');

for fi = 1:n_files
    fpath = fullfile(csv_files(fi).folder, csv_files(fi).name);
    fname = csv_files(fi).name;
    parts = strsplit(csv_files(fi).folder, filesep);
    group = parts{end};

    fprintf('[%3d/%d] %s / %s\n', fi, n_files, group, fname);
```

```
    try
        raw = readmatrix(fpath, 'NumHeaderLines', 1);
    catch ME
        fprintf('  SKIP — %s\n', ME.message); continue
    end

    I1 = raw(:,8);  I4 = raw(:,11);  N = length(I1);
    if N < WIN_TRANS*2 + BUFFER*2
        fprintf('  SKIP — too few samples\n'); continue
    end

    % --- Transition ---
    trans_row   = findTransition(I1, WIN_TRANS);
    healthy_end = trans_row - BUFFER;
    fault_start = trans_row + BUFFER;
    if healthy_end < WIN_TRANS+1 || fault_start > N-RMS_WIN
        fprintf('  SKIP — transition at edge\n'); continue
    end

    % --- EFFECTIVE_RATIO ---
    cal_len   = CAL_CYCLES * SAMPLES_PER_CYCLE;
    cal_start = max(1, healthy_end - cal_len + 1);
    rms_p_cal = sqrt(mean(I1(cal_start:healthy_end).^2));
    rms_s_cal = sqrt(mean(I4(cal_start:healthy_end).^2));
    if rms_p_cal < TR_MIN_CURRENT || rms_s_cal < TR_MIN_CURRENT
        ER = 1.0;
    else
        ER = rms_p_cal / rms_s_cal;
    end

    % --- Windows ---
    h_starts = (WIN_TRANS+1) : RMS_WIN : (healthy_end - RMS_WIN + 1);
    f_starts = fault_start   : RMS_WIN : (N - RMS_WIN + 1);
    all_starts = [h_starts, f_starts];
    all_zones  = [repmat({'H'},1,numel(h_starts)),
repmat({'F'},1,numel(f_starts))];

    for wi = 1:numel(all_starts)
        ws = all_starts(wi);
        we = ws + RMS_WIN - 1;

        rms_p  = sqrt(mean(I1(ws:we).^2));
        rms_s  = sqrt(mean(I4(ws:we).^2));
        p_comp = rms_p / ER;
        i_diff = abs(p_comp - rms_s);
        i_bias = (p_comp + rms_s) / 2;
        zone   = all_zones{wi};

        if strcmp(zone, 'H')
            all_ib_h(end+1) = i_bias;  %#ok<AGROW>
            all_id_h(end+1) = i_diff;  %#ok<AGROW>
        else
            all_ib_f(end+1) = i_bias;  %#ok<AGROW>
            all_id_f(end+1) = i_diff;  %#ok<AGROW>
        end

        fprintf(fid_raw, '%s,%s,%d,%s,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f\n', ...
            fname, group, wi, zone, rms_p, rms_s, ER, p_comp, i_diff, i_bias);
```

```
    end
end
fclose(fid_raw);

fprintf('\nTotal windows — Healthy: %d   Fault: %d\n\n', numel(all_ib_h),
numel(all_ib_f));

% ========================================================================
% 6.  SCATTER PLOT
% ========================================================================
fprintf('--- Generating scatter plot ---\n');

ib_range = [0, max([all_ib_h, all_ib_f]) * 1.05];
ib_curve = linspace(0, ib_range(2), 500);

% Characteristic curve from TUNING PARAMETERS
id_curve = arrayfun(@(ib) dualSlopeThr(ib, SLOPE1_PLOT, PICKUP_PLOT, KNEE_PLOT,
...
                                          SLOPE2_PLOT, OFFSET2_PLOT), ib_curve);

fig = figure('Visible','off','Position',[0 0 1100 650]);
hold on; grid on; box on;

% Scatter: Healthy (blue circles) and Fault (red crosses)

scatter(all_ib_h, all_id_h, 20, [0.2 0.4 0.8], 'o', 'DisplayName','Healthy
windows');
scatter(all_ib_f, all_id_f, 30, [0.85 0.1 0.1], 'x', 'DisplayName','Fault
windows');

% Characteristic curve
plot(ib_curve, id_curve, 'k-', 'LineWidth', 2.0, 'DisplayName', ...
    sprintf('Dual-Slope (S1=%.2f, S2=%.2f, Pu=%.2f, Knee=%.1f)', ...
    SLOPE1_PLOT, SLOPE2_PLOT, PICKUP_PLOT, KNEE_PLOT));

% Knee point marker
knee_id = dualSlopeThr(KNEE_PLOT, SLOPE1_PLOT, PICKUP_PLOT, KNEE_PLOT,
SLOPE2_PLOT, OFFSET2_PLOT);
plot(KNEE_PLOT, knee_id, 'ko', 'MarkerSize', 8, 'MarkerFaceColor','k', ...
    'HandleVisibility','off');
text(KNEE_PLOT + 0.3, knee_id, sprintf('Knee (%.0fA, %.2fA)', KNEE_PLOT, knee_id),
...
    'FontSize', 9);

% TRIP / SAFE labels
text(ib_range(2)*0.65, min(id_curve)*0.3, 'SAFE (No Trip)', ...
    'FontSize', 12, 'Color', [0.2 0.5 0.2], 'FontWeight', 'bold');
text(ib_range(2)*0.1, max(id_curve)*0.8, 'TRIP Zone', ...
    'FontSize', 12, 'Color', [0.7 0 0], 'FontWeight', 'bold');

xlabel('I_{bias} (A)', 'FontSize', 12);
ylabel('I_{diff} (A)', 'FontSize', 12);
title(sprintf('Differential Bias Characteristic — %d Healthy + %d Fault windows
(%d files)', ...
    numel(all_ib_h), numel(all_ib_f), n_files), 'FontSize', 12);
legend('Location','northwest','FontSize', 9);

xlim(ib_range);
ylim([0, max([all_id_h, all_id_f]) * 1.10]);
```

```
out_scatter = fullfile(out_dir, 'bias_scatter.png');
exportgraphics(fig, out_scatter, 'Resolution', 150);
close(fig);
fprintf('Scatter plot saved: %s\n\n', out_scatter);

% ========================================================================
% 7.  GRID SEARCH
% ========================================================================
fprintf('--- Pass 2: Grid Search ---\n');

% Build combined dataset for fast scoring
ib_all = [all_ib_h, all_ib_f];
id_all = [all_id_h, all_id_f];
zone_h = [true(1, numel(all_ib_h)), false(1, numel(all_ib_f))];
zone_f = ~zone_h;

n_h = sum(zone_h);  n_f = sum(zone_f);

% Result storage: [pickup, s1, s2, knee, fp_pct, det_pct, min_h_margin,
min_f_margin]
results = [];

total_combos = numel(gs_pickup)*numel(gs_slope1)*numel(gs_slope2)*numel(gs_knee);
combo_count  = 0;

for pu = gs_pickup
    for s1 = gs_slope1
        for s2 = gs_slope2
            if s2 <= s1, continue; end   % slope2 must exceed slope1 (physically)
            for kn = gs_knee
                combo_count = combo_count + 1;
                off2 = (s2 - s1) * kn - pu;  % continuity condition

                % Vectorised threshold computation
                thr = zeros(1, numel(ib_all));
                lo  = ib_all < kn;
                thr( lo) = s1 * ib_all( lo) + pu;
                thr(~lo) = s2 * ib_all(~lo) - off2;

                trip = id_all > thr;

                fp_count  = sum(trip & zone_h);
                det_count = sum(trip & zone_f);

                fp_pct  = 100 * fp_count  / n_h;
                det_pct = 100 * det_count / n_f;

                % Margins (positive = good)
                margin_h = min(thr(zone_h) - id_all(zone_h));   % healthy: want
thr > id → positive
                margin_f = min(id_all(zone_f) - thr(zone_f));   % fault:   want id
> thr → positive

                results(end+1, :) = [pu, s1, s2, kn, fp_pct, det_pct, margin_h,
margin_f]; %#ok<AGROW>
            end
        end
    end
```

```
end

fprintf('Evaluated %d/%d valid combinations.\n', size(results,1), total_combos);

% --- Filter: FP=0 AND Detection=100 ---
pass_mask = (results(:,5) == 0) & (results(:,6) == 100);
n_pass    = sum(pass_mask);
fprintf('Combinations with FP=0%% AND Det=100%%: %d\n\n', n_pass);

% --- Sort passing results by composite margin (min of H and F margin) ---
pass_results = results(pass_mask, :);
if ~isempty(pass_results)
    composite_margin = min(pass_results(:,7), pass_results(:,8));
    [~, sort_idx]    = sort(composite_margin, 'descend');
    pass_results     = pass_results(sort_idx, :);
end

% --- Sort ALL results by detection rate desc, then fp asc, then margin ---
all_sort_key = results(:,6) - results(:,5)/100;
[~, all_sort] = sort(all_sort_key, 'descend');
results_sorted = results(all_sort, :);

% ========================================================================
% 8.  WRITE GRID SEARCH CSV
% ========================================================================
fid_gs = fopen(fullfile(out_dir, 'grid_search_results.csv'), 'w');
fprintf(fid_gs,
'PICKUP,SLOPE1,SLOPE2,KNEE_POINT,OFFSET2,FP_pct,Det_pct,MinMargin_H,MinMargin_F,Co
mpositeMargin,PASS\n');

for k = 1:size(results_sorted,1)
    r    = results_sorted(k,:);
    off2 = (r(3)-r(2))*r(4) - r(1);
    comp = min(r(7), r(8));
    pass_str = ternary((r(5)==0) && (r(6)==100), 'PASS', 'FAIL');
    fprintf(fid_gs, '%.3f,%.3f,%.3f,%.1f,%.4f,%.2f,%.2f,%.4f,%.4f,%.4f,%s\n', ...
        r(1),r(2),r(3),r(4), off2, r(5),r(6), r(7),r(8), comp, pass_str);
end
fclose(fid_gs);
fprintf('Grid search CSV saved: %s\n',
fullfile(out_dir,'grid_search_results.csv'));

% ========================================================================
% 9.  WRITE BEST PARAMETERS (firmware-ready)
% ========================================================================
fid_best = fopen(fullfile(out_dir, 'grid_search_best.txt'), 'w');

if ~isempty(pass_results)
    best = pass_results(1,:);
    off2_best = (best(3)-best(2))*best(4) - best(1);

    fprintf(fid_best, '=== Grid Search Best Result ===\n');
    fprintf(fid_best, 'FP=0%%%% AND Detection=100%%%% passing sets: %d\n\n',
n_pass);
    fprintf(fid_best, 'Best (max composite margin = %.4f A):\n',
min(best(7),best(8)));
    fprintf(fid_best, '  PICKUP     = %.3f A\n', best(1));
    fprintf(fid_best, '  SLOPE1     = %.3f\n',   best(2));
    fprintf(fid_best, '  SLOPE2     = %.3f\n',   best(3));
```

```
    fprintf(fid_best, '  KNEE_POINT = %.1f A\n', best(4));
    fprintf(fid_best, '  OFFSET2    = %.4f A  (calculated: (S2-S1)*Knee -
Pickup)\n', off2_best);
    fprintf(fid_best, '  Margin_H   = %.4f A  (healthy safety margin)\n',
best(7));
    fprintf(fid_best, '  Margin_F   = %.4f A  (fault detection margin)\n',
best(8));
    fprintf(fid_best, '\n--- Copy to firmware (.ino) ---\n');
    fprintf(fid_best, 'const float SLOPE1     = %.3ff;\n', best(2));
    fprintf(fid_best, 'const float PICKUP     = %.3ff;\n', best(1));
    fprintf(fid_best, 'const float KNEE_POINT = %.1ff;\n', best(4));
    fprintf(fid_best, 'const float SLOPE2     = %.3ff;\n', best(3));
    fprintf(fid_best, 'const float OFFSET2    = %.4ff;\n', off2_best);

    % Also print to console
    fprintf('\n');
    fprintf('╔═══════════════════════════════════════════════════╗\n');
    fprintf('║         BEST PARAMETERS (FP=0, Det=100)         ╒\n');
    fprintf('╚═══════════════════════════════════════════════════╝\n');
    fprintf('  PICKUP     = %.3f A                              \n', best(1));
    fprintf('  SLOPE1     = %.3f                                \n', best(2));
    fprintf('  SLOPE2     = %.3f                                \n', best(3));
    fprintf('  KNEE_POINT = %.1f A                              \n', best(4));
    fprintf('  OFFSET2    = %.4f A                              \n', off2_best);
    fprintf('  Margin_H   = %.4f A                              \n', best(7));
    fprintf('  Margin_F   = %.4f A                              \n', best(8));
    fprintf('  Passing sets: %-4d / %-4d evaluated              \n', n_pass,
total_combos);
    fprintf('                                                   \n\n');
else
    fprintf(fid_best, 'NO combination achieved FP=0%% AND Detection=100%%.\n');
    fprintf(fid_best, 'Top 5 by detection rate:\n\n');
    for k = 1:min(5, size(results_sorted,1))
        r    = results_sorted(k,:);
        off2 = (r(3)-r(2))*r(4) - r(1);
        fprintf(fid_best, '  FP=%.1f%% Det=%.1f%%  PU=%.3f S1=%.3f S2=%.3f
Knee=%.1f OFFSET2=%.4f\n', ...
            r(5),r(6), r(1),r(2),r(3),r(4), off2);
    end
    fprintf('\n[WARNING] No perfect combination found — consider widening grid
search range.\n\n');
end
fclose(fid_best);
fprintf('Best parameters saved: %s\n', fullfile(out_dir,'grid_search_best.txt'));

fprintf('\n=== Done. All outputs in %s/ ===\n', out_dir);
fprintf('  bias_characteristic.csv   — raw I_diff/I_bias per window\n');
fprintf('  bias_scatter.png          — scatter plot (tune SECTION 1 and re-run to
adjust curve)\n');
fprintf('  grid_search_results.csv   — all evaluated combinations\n');
fprintf('  grid_search_best.txt      — firmware-ready best parameters\n');

% ========================================================================
% LOCAL FUNCTIONS
% ========================================================================

function row = findTransition(I1, win)
N = length(I1);
best_row = win+1;  best_ratio = 0;
```

```
for i = win+1 : N-win
    rb = sqrt(mean(I1(i-win : i-1).^2));
    ra = sqrt(mean(I1(i     : i+win-1).^2));
    r  = (rb > 1e-9) * ra / max(rb, 1e-9);
    if r > best_ratio, best_ratio = r; best_row = i; end
end
row = best_row;
end

function thr = dualSlopeThr(ib, s1, pu, knee, s2, off2)
if ib < knee
    thr = s1 * ib + pu;
else
    thr = s2 * ib - off2;
end
end

function s = ternary(cond, a, b)
if cond, s = a; else, s = b; end
end
```

สคริปต์ script_3_simulate_relay.m

```
% ========================================================================
% simulate_relay.m
% Simulation of differential_relay_v9.ino on recorded CSV signals
%
% Faithfully replicates firmware v9 logic:
%   Boot  : EFFECTIVE_RATIO = mean(I_pri_phases) / mean(I_sec_phases)
%           averaged over Healthy zone (mirrors runEffectiveRatioCalibration)
%   Loop  : per RMS-window (40 ms = 800 samples @20 kHz)
%             i_pri_comp = I_pri_rms / EFFECTIVE_RATIO
%             i_diff     = |i_pri_comp - I_sec_rms|
%             i_bias     = (i_pri_comp + I_sec_rms) / 2
%             trip       = shouldTrip(i_diff, i_bias)   [Dual Slope]
%
% Dataset assumptions
%   - CSV cols: EventNo,Date,Time,U1,U2,U3,U4,I1,I2,I3,I4
%   - I1 (col 8)  = Primary  line current  [A, already calibrated by Hioki]
%   - I4 (col 11) = Secondary line current [A]
%   - fs = 20 000 Hz,  f0 = 50 Hz
%   - 5 600 rows per file
%   - Transition Healthy→Fault located via sliding-RMS window on I1
%
% Outputs
%   Simulation_Output/simulation_results.csv  — per-file summary
%   Simulation_Output/simulation_log.csv      — per-window time series
% ========================================================================

clear; clc;

% ========================================================================
% 1.  FIRMWARE PARAMETERS  (direct copy from differential_relay_v9.ino)
% ========================================================================
FS             = 20000;   % Hz — Hioki PQ3198 recording rate
F0             = 50;      % Hz — supply fundamental
PROTECTION_MODE = 1;      % 1 = Dual Slope (IEC), 2 = Single Slope

% Dual Slope (PROTECTION_MODE == 1)
SLOPE1         = 0.10;
PICKUP         = 0.45;    % A  — MinPickup
KNEE_POINT     = 5.0;    % A  — knee Ibias
SLOPE2         = 0.50;
OFFSET2        = 1.55;    % A  — intercept correction for slope-2 line

% Single Slope (PROTECTION_MODE == 2)
SLOPE_S2       = 0.60;
PICKUP_S2      = 10.0;

% Calibration — ER averaged over how many healthy cycles
CAL_CYCLES     = 10;      % match TR_MEASURE_MS/cycle_len ≈ 500 ms / 1 cy
TR_MIN_CURRENT = 0.5;     % A — fallback trigger

% RMS window size (firmware: 40 ms accumulation @ 1 ms sampling)
% Hioki @ 20 kHz → 40 ms = 800 samples
SAMPLES_PER_CYCLE = FS / F0;           % 400
RMS_WIN           = round(0.040 * FS); % 800 samples  (= 2 cycles)

% ========================================================================
% 2.  shouldTrip() — exact translation of firmware function
```

```
% ========================================================================
shouldTrip = @(id, ib) tripLogic(id, ib, PROTECTION_MODE, ...
    SLOPE1, PICKUP, KNEE_POINT, SLOPE2, OFFSET2, SLOPE_S2, PICKUP_S2);

% ========================================================================
% 3.  TRANSITION DETECTION — sliding RMS window on I1 (win = 200 samples)
% ========================================================================
WIN_TRANS  = 200;   % samples — same as README step 4
BUFFER     = 300;   % samples — exclude transient around transition

% ========================================================================
% 4.  FIND FILES
% ========================================================================
base_dir = 'Selected_Signals';
if ~isfolder(base_dir)
    error('Folder "%s" not found. Run from the project root.', base_dir);
end

csv_files = dir(fullfile(base_dir, '**', '*.csv'));
n_files   = numel(csv_files);
if n_files == 0
    error('No CSV files found under %s', base_dir);
end
fprintf('Found %d CSV files.\n', n_files);

% ========================================================================
% 5.  OUTPUT SETUP
% ========================================================================
out_dir = 'Simulation_Output';
if ~isfolder(out_dir), mkdir(out_dir); end

fid_res = fopen(fullfile(out_dir, 'simulation_results.csv'), 'w');
fprintf(fid_res, ['File,Group,TransitionRow,ER,' ...
    'Healthy_FP_pct,Healthy_windows,' ...
    'Fault_Det_pct,Fault_windows,' ...
    'I_diff_healthy_mean,I_diff_fault_mean,' ...
    'PASS\n']);

fid_log = fopen(fullfile(out_dir, 'simulation_log.csv'), 'w');
fprintf(fid_log, ['File,WindowIdx,TimeSec,Zone,' ...
    'I_pri_rms,I_sec_rms,I_pri_comp,I_diff,I_bias,Threshold,Trip\n']);

% ========================================================================
% 6.  MAIN LOOP — one file at a time
% ========================================================================
for fi = 1:n_files
    fpath = fullfile(csv_files(fi).folder, csv_files(fi).name);
    fname = csv_files(fi).name;
    % derive group name from parent folder
    parts = strsplit(csv_files(fi).folder, filesep);
    group = parts{end};

    fprintf('[%3d/%d] %s / %s\n', fi, n_files, group, fname);

    % ------------------------------------------------------------------
    % 6a.  Load CSV
    % ------------------------------------------------------------------
    try
        raw = readmatrix(fpath, 'NumHeaderLines', 1);
```

```
catch ME
    fprintf('  SKIP — cannot read file: %s\n', ME.message);
    continue
end

I1 = raw(:, 8);   % Primary  current [A]
I4 = raw(:, 11);  % Secondary current [A]
N  = length(I1);

if N < WIN_TRANS * 2 + BUFFER * 2
    fprintf('  SKIP — too few samples (%d)\n', N);
    continue
end

% ------------------------------------------------------------------
% 6b.  Find transition row (sliding RMS on I1, win=200)
% ------------------------------------------------------------------
trans_row = findTransition(I1, WIN_TRANS);

% Zones (1-indexed)
healthy_end = trans_row - BUFFER;
fault_start  = trans_row + BUFFER;

if healthy_end < WIN_TRANS + 1 || fault_start > N - RMS_WIN
    fprintf('  SKIP — transition too close to edge (row %d)\n', trans_row);
    continue
end

% ------------------------------------------------------------------
% 6c.  EFFECTIVE_RATIO calibration — firmware boot step
%       Mirror: measureRMSBlocking over CAL_CYCLES healthy cycles,
%               then ER = mean(I_pri_rms_phases) / mean(I_sec_rms_phases)
%       Here we only have Phase A primary (I1) and Phase A secondary (I4)
%       → ER = RMS(I1_healthy_last_CAL) / RMS(I4_healthy_last_CAL)
% ------------------------------------------------------------------
cal_len   = CAL_CYCLES * SAMPLES_PER_CYCLE;
cal_start = max(1, healthy_end - cal_len + 1);
cal_end   = healthy_end;

i_pri_cal = sqrt(mean(I1(cal_start:cal_end).^2));
i_sec_cal = sqrt(mean(I4(cal_start:cal_end).^2));

if i_pri_cal < TR_MIN_CURRENT || i_sec_cal < TR_MIN_CURRENT
    ER = 1.0;   % firmware fallback
    fprintf('  WARN — low calibration current (%.3fA / %.3fA) → ER=1.0\n', ...
        i_pri_cal, i_sec_cal);
else
    ER = i_pri_cal / i_sec_cal;
end
fprintf('  trans_row=%d  ER=%.4f\n', trans_row, ER);

% ------------------------------------------------------------------
% 6d.  Per-window RMS + protection decision
%       Firmware accumulates for 40 ms (800 smp) then evaluates.
%       We step through the signal in non-overlapping 800-sample windows.
% ------------------------------------------------------------------
% Build window start indices for Healthy and Fault zones separately
h_starts = (WIN_TRANS + 1) : RMS_WIN : (healthy_end - RMS_WIN + 1);
f_starts = fault_start    : RMS_WIN : (N - RMS_WIN + 1);
```

```
    all_starts = [h_starts, f_starts];
    all_zones  = [repmat({'H'}, 1, numel(h_starts)), ...
                  repmat({'F'}, 1, numel(f_starts))];

    n_h_trip = 0;   n_h_win = numel(h_starts);
    n_f_trip = 0;   n_f_win = numel(f_starts);
    id_h_acc = [];
    id_f_acc = [];

    for wi = 1:numel(all_starts)
        ws = all_starts(wi);
        we = ws + RMS_WIN - 1;

        seg_pri = I1(ws:we);
        seg_sec = I4(ws:we);

        i_pri_rms  = sqrt(mean(seg_pri.^2));
        i_sec_rms  = sqrt(mean(seg_sec.^2));

        i_pri_comp = i_pri_rms / ER;
        i_diff_val = abs(i_pri_comp - i_sec_rms);
        i_bias_val = (i_pri_comp + i_sec_rms) / 2;

        trip = shouldTrip(i_diff_val, i_bias_val);
        thr  = computeThreshold(i_bias_val, PROTECTION_MODE, ...
                   SLOPE1, PICKUP, KNEE_POINT, SLOPE2, OFFSET2, ...
                   SLOPE_S2, PICKUP_S2);

        zone = all_zones{wi};
        t_sec = ws / FS;

        % Accumulate
        if strcmp(zone, 'H')
            if trip, n_h_trip = n_h_trip + 1; end
            id_h_acc(end+1) = i_diff_val; %#ok<AGROW>
        else
            if trip, n_f_trip = n_f_trip + 1; end
            id_f_acc(end+1) = i_diff_val; %#ok<AGROW>
        end

        % Write log row
        fprintf(fid_log, '%s,%s,%d,%.4f,%s,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f,%d\n',
...
            fname, group, wi, t_sec, zone, ...
            i_pri_rms, i_sec_rms, i_pri_comp, i_diff_val, i_bias_val, thr, trip);
    end

    % ------------------------------------------------------------------
    % 6e.  Score
    % ------------------------------------------------------------------
    fp_pct  = (n_h_win > 0) * 100 * n_h_trip / max(n_h_win, 1);
    det_pct = (n_f_win > 0) * 100 * n_f_trip / max(n_f_win, 1);
    pass    = (n_h_trip == 0) && (n_f_trip == n_f_win) && (n_f_win > 0);

    id_h_mean = nanmean_safe(id_h_acc);
    id_f_mean = nanmean_safe(id_f_acc);

    fprintf('  H_windows=%d  FP=%.1f%%  |  F_windows=%d  Det=%.1f%%  |  %s\n', ...
```

```
        n_h_win, fp_pct, n_f_win, det_pct, ternary(pass,'PASS','FAIL'));

    % Write results row
    fprintf(fid_res, '%s,%s,%d,%.4f,%.2f,%d,%.2f,%d,%.4f,%.4f,%s\n', ...
        fname, group, trans_row, ER, ...
        fp_pct, n_h_win, det_pct, n_f_win, ...
        id_h_mean, id_f_mean, ternary(pass,'PASS','FAIL'));
end

fclose(fid_res);
fclose(fid_log);
fprintf('\nDone. Results in %s/\n', out_dir);

% ========================================================================
% LOCAL FUNCTIONS
% ========================================================================

function row = findTransition(I1, win)
% Returns the sample index with the maximum before/after RMS ratio.
N = length(I1);
best_row   = win + 1;
best_ratio = 0;
for i = win + 1 : N - win
    r_before = sqrt(mean(I1(i-win : i-1).^2));
    r_after  = sqrt(mean(I1(i     : i+win-1).^2));
    if r_before > 0
        r = r_after / r_before;
    else
        r = 0;
    end
    if r > best_ratio
        best_ratio = r;
        best_row   = i;
    end
end
row = best_row;
end

% ------------------------------------------------------------------------
function trip = tripLogic(id, ib, mode, ...
    slope1, pickup, knee, slope2, offset2, slope_s2, pickup_s2)
if mode == 1
    if ib < knee
        thr = slope1 * ib + pickup;
    else
        thr = slope2 * ib - offset2;
    end
    trip = id > thr;
else
    trip = id > slope_s2 * ib + pickup_s2;
end
end

% ------------------------------------------------------------------------
function thr = computeThreshold(ib, mode, ...
    slope1, pickup, knee, slope2, offset2, slope_s2, pickup_s2)
if mode == 1
    if ib < knee
        thr = slope1 * ib + pickup;
```

```
    else
        thr = slope2 * ib - offset2;
    end
else
    thr = slope_s2 * ib + pickup_s2;
end
end

% ---------------------------------------------------------------------------
function v = nanmean_safe(arr)
if isempty(arr)
    v = NaN;
else
    v = mean(arr(~isnan(arr)));
end
end

% ---------------------------------------------------------------------------
function s = ternary(cond, a, b)
if cond, s = a; else, s = b; end
end
```

สคริปต์ script_4_simulate_relay_worstcase_noise.m

```
% ========================================================================
% simulate_relay_worstcase_noise.m
% Worst-Case Noise Simulation — based on simulate_relay.m
%
% เพิ่ม noise injection จาก hardware noise budget (README Section 5):
%
%   I_diff_noise(I_bias) = ADC_CONTRIBUTION + CT_SLOPE × I_bias
%     ADC_CONTRIBUTION  = 2 LSB × CT_RATIO × R_BURDEN = 0.0160 A
%     CT_SLOPE          = 2 × (CT_ratio_err + CT_phase_err) ≈ 0.026
%   → worst-case I_diff_noise ≈ 0.016 + 0.026 × I_bias  [A]
%
% Noise injection strategy (additive, worst direction):
%   I_pri_noisy = seg_pri × (1 + CT_ERROR_FRAC) + ADC_OFFSET_A  [+DC +gain]
%   I_sec_noisy = seg_sec × (1 - CT_ERROR_FRAC) - ADC_OFFSET_A  [-DC -gain]
%   → I_diff_noisy increases by ≈ ADC_CONTRIBUTION + CT_SLOPE × I_bias
%
% Outputs (all in Simulation_Output/):
%   simulation_log_noisy.csv     — per-window: clean vs noisy I_diff + margin
%   simulation_results_noisy.csv — per-file: PASS/FAIL clean vs noisy
%   + console summary: worst-margin files, noise budget stats
%
% Parameters และ firmware logic เหมือน simulate_relay.m ทุกอย่าง
% เปลี่ยนได้เฉพาะ Section 2 (NOISE BUDGET PARAMETERS)
% ========================================================================

clear; clc;

% ========================================================================
% 1.  FIRMWARE PARAMETERS  (sync กับ differential_relay_v9.ino)
% ========================================================================
FS              = 20000;
F0              = 50;
PROTECTION_MODE = 1;

% Dual Slope (PROTECTION_MODE == 1)
SLOPE1         = 0.10;
PICKUP         = 0.45;    % A  — MinPickup
KNEE_POINT     = 5.0;    % A  — knee Ibias
SLOPE2         = 0.50;
OFFSET2        = 1.55;    % A  — intercept correction for slope-2 line

% Single Slope
SLOPE_S2        = 0.60;
PICKUP_S2       = 10.0;

% Calibration
CAL_CYCLES      = 10;
TR_MIN_CURRENT  = 0.5;

% RMS window
SAMPLES_PER_CYCLE = FS / F0;            % 400
RMS_WIN           = round(0.040 * FS);  % 800 samples

% ========================================================================
% 2.  NOISE BUDGET PARAMETERS  (แก้ได้ที่นี่เท่านั้น)
% ========================================================================
```

```
% -- ADC noise (systematic DC offset, worst-case 2 LSB) --
ADC_LSB_V       = 3.3 / 4096;      % ≈ 0.000806 V per LSB
ADC_OFFSET_LSB  = 2;                % worst-case systematic offset (LSB)
CT_RATIO_HW     = 60.0;             % A/V  (30A/5A ÷ 0.10 Ω)
ADC_OFFSET_A    = ADC_OFFSET_LSB * ADC_LSB_V * CT_RATIO_HW;
% → 2 × 0.000806 × 60 ≈ 0.0097 A  (per channel, opposite direction)

% -- CT Class 0.5 error --
CT_RATIO_ERR    = 0.005;            % ±0.5% ratio error per CT
CT_PHASE_ERR_DEG= 0.5 / 60;        % 30 arcmin → degrees  (≈ 0.00833°)
CT_PHASE_ERR_RAD= CT_PHASE_ERR_DEG * pi / 180;
% Combined worst-case fraction: ratio + phase (orthogonal approx)
CT_ERROR_FRAC   = sqrt(CT_RATIO_ERR^2 + sin(CT_PHASE_ERR_RAD)^2);
% ≈ 0.0130  (1.30%)

% Analytical noise floor for console report
noise_fn = @(ib) 2*ADC_OFFSET_A + 2*CT_ERROR_FRAC*ib;
% (×2 because Pri and Sec errors add in worst direction)

fprintf('=== WORST-CASE NOISE PARAMETERS ===\n');
fprintf('ADC offset per channel : %.4f A  (%d LSB)\n', ADC_OFFSET_A,
ADC_OFFSET_LSB);
fprintf('CT error fraction      : %.4f  (%.2f%%)\n', CT_ERROR_FRAC,
CT_ERROR_FRAC*100);
fprintf('Noise floor formula    : %.4f + %.4f × I_bias  [A]\n', 2*ADC_OFFSET_A,
2*CT_ERROR_FRAC);
fprintf('At I_bias=3.2 A        : %.4f A  (README: ~0.099 A)\n', noise_fn(3.2));
fprintf('====================================\n\n');

% ========================================================================
% 3.  shouldTrip + computeThreshold helpers
% ========================================================================
shouldTrip = @(id, ib) tripLogic(id, ib, PROTECTION_MODE, ...
    SLOPE1, PICKUP, KNEE_POINT, SLOPE2, OFFSET2, SLOPE_S2, PICKUP_S2);

% ========================================================================
% 4.  TRANSITION DETECTION PARAMETERS
% ========================================================================
WIN_TRANS = 200;
BUFFER    = 300;

% ========================================================================
% 5.  FIND FILES
% ========================================================================
base_dir = 'Selected_Signals';
if ~isfolder(base_dir)
    error('Folder "%s" not found. Run from project root.', base_dir);
end
csv_files = dir(fullfile(base_dir, '**', '*.csv'));
n_files   = numel(csv_files);
if n_files == 0, error('No CSV files found under %s', base_dir); end
fprintf('Found %d CSV files.\n\n', n_files);

% ========================================================================
% 6.  OUTPUT SETUP
% ========================================================================
out_dir = 'Simulation_Output';
if ~isfolder(out_dir), mkdir(out_dir); end
```

```
% -- Log file (per-window) --
fid_log = fopen(fullfile(out_dir, 'simulation_log_noisy.csv'), 'w');
fprintf(fid_log, ['File,Group,WindowIdx,TimeSec,Zone,' ...
    'I_pri_rms,I_sec_rms,' ...
    'I_diff_clean,I_diff_noisy,' ...
    'I_bias_clean,I_bias_noisy,' ...
    'Threshold,' ...
    'Noise_budget,' ...
    'Margin_clean,Margin_noisy,' ...
    'Trip_clean,Trip_noisy\n']);

% -- Results file (per-file) --
fid_res = fopen(fullfile(out_dir, 'simulation_results_noisy.csv'), 'w');
fprintf(fid_res, ['File,Group,TransitionRow,ER,' ...
    'H_windows,FP_clean,FP_noisy,' ...
    'F_windows,Det_clean,Det_noisy,' ...
    'MinMargin_H_clean,MinMargin_H_noisy,' ...
    'MinMargin_F_clean,MinMargin_F_noisy,' ...
    'PASS_clean,PASS_noisy\n']);

% ========================================================================
% 7.  MAIN LOOP
% ========================================================================
% Accumulators for final console summary
summary = struct();
summary_idx = 0;

for fi = 1:n_files
    fpath = fullfile(csv_files(fi).folder, csv_files(fi).name);
    fname = csv_files(fi).name;
    parts = strsplit(csv_files(fi).folder, filesep);
    group = parts{end};

    fprintf('[%3d/%d] %s / %s\n', fi, n_files, group, fname);

    % ------------------------------------------------------------------
    % 7a.  Load CSV
    % ------------------------------------------------------------------
    try
        raw = readmatrix(fpath, 'NumHeaderLines', 1);
    catch ME
        fprintf('  SKIP — %s\n', ME.message);
        continue
    end
    I1 = raw(:, 8);
    I4 = raw(:, 11);
    N  = length(I1);
    if N < WIN_TRANS*2 + BUFFER*2
        fprintf('  SKIP — too few samples\n');
        continue
    end

    % ------------------------------------------------------------------
    % 7b.  Find transition
    % ------------------------------------------------------------------
    trans_row   = findTransition(I1, WIN_TRANS);
    healthy_end = trans_row - BUFFER;
    fault_start = trans_row + BUFFER;
    if healthy_end < WIN_TRANS+1 || fault_start > N-RMS_WIN
```

```
    fprintf('  SKIP — transition too close to edge\n');
    continue
end

% ------------------------------------------------------------------
% 7c.  EFFECTIVE_RATIO calibration (clean signal — mirrors firmware boot)
% ------------------------------------------------------------------
cal_len   = CAL_CYCLES * SAMPLES_PER_CYCLE;
cal_start = max(1, healthy_end - cal_len + 1);
i_pri_cal = sqrt(mean(I1(cal_start:healthy_end).^2));
i_sec_cal = sqrt(mean(I4(cal_start:healthy_end).^2));
if i_pri_cal < TR_MIN_CURRENT || i_sec_cal < TR_MIN_CURRENT
    ER = 1.0;
else
    ER = i_pri_cal / i_sec_cal;
end
fprintf('  trans_row=%d  ER=%.4f\n', trans_row, ER);

% ------------------------------------------------------------------
% 7d.  Window indices
% ------------------------------------------------------------------
h_starts  = (WIN_TRANS+1) : RMS_WIN : (healthy_end - RMS_WIN + 1);
f_starts  = fault_start   : RMS_WIN : (N - RMS_WIN + 1);
all_starts = [h_starts, f_starts];
all_zones  = [repmat({'H'}, 1, numel(h_starts)), ...
              repmat({'F'}, 1, numel(f_starts))];

% Counters
n_h_win = numel(h_starts);
n_f_win = numel(f_starts);
n_h_trip_clean = 0;  n_h_trip_noisy = 0;
n_f_trip_clean = 0;  n_f_trip_noisy = 0;

% Margin tracking
min_margin_H_clean =  Inf;  min_margin_H_noisy =  Inf;
min_margin_F_clean =  Inf;  min_margin_F_noisy =  Inf;

% ------------------------------------------------------------------
% 7e.  Per-window loop
% ------------------------------------------------------------------
for wi = 1:numel(all_starts)
    ws = all_starts(wi);
    we = ws + RMS_WIN - 1;

    seg_pri = I1(ws:we);
    seg_sec = I4(ws:we);

    % === CLEAN ===
    i_pri_rms_c  = sqrt(mean(seg_pri.^2));
    i_sec_rms_c  = sqrt(mean(seg_sec.^2));
    i_pri_comp_c = i_pri_rms_c / ER;
    i_diff_c     = abs(i_pri_comp_c - i_sec_rms_c);
    i_bias_c     = (i_pri_comp_c + i_sec_rms_c) / 2;
    thr_c        = computeThreshold(i_bias_c, PROTECTION_MODE, ...
                       SLOPE1, PICKUP, KNEE_POINT, SLOPE2, OFFSET2, ...
                       SLOPE_S2, PICKUP_S2);
    trip_c       = shouldTrip(i_diff_c, i_bias_c);

    % === NOISY — worst-case injection ===
```

```
        % Pri: +CT_gain error, +ADC offset  → RMS increases
        % Sec: -CT_gain error, -ADC offset  → RMS decreases
        % Result: i_diff_noisy > i_diff_clean (worst direction)
        seg_pri_n = seg_pri * (1 + CT_ERROR_FRAC) + ADC_OFFSET_A;
        seg_sec_n = seg_sec * (1 - CT_ERROR_FRAC) - ADC_OFFSET_A;

        i_pri_rms_n  = sqrt(mean(seg_pri_n.^2));
        i_sec_rms_n  = sqrt(mean(seg_sec_n.^2));
        i_pri_comp_n = i_pri_rms_n / ER;
        i_diff_n     = abs(i_pri_comp_n - i_sec_rms_n);
        i_bias_n     = (i_pri_comp_n + i_sec_rms_n) / 2;
        thr_n        = computeThreshold(i_bias_n, PROTECTION_MODE, ...
                           SLOPE1, PICKUP, KNEE_POINT, SLOPE2, OFFSET2, ...
                           SLOPE_S2, PICKUP_S2);
        trip_n       = shouldTrip(i_diff_n, i_bias_n);

        % Noise budget at this operating point
        noise_budget = noise_fn(i_bias_c);

        % Margins  (positive = safe gap from threshold)
        %   Healthy zone: want I_diff < threshold → margin = threshold - I_diff
        %   Fault zone  : want I_diff > threshold → margin = I_diff - threshold
        zone = all_zones{wi};
        if strcmp(zone, 'H')
            margin_c = thr_c - i_diff_c;
            margin_n = thr_n - i_diff_n;
            if trip_c, n_h_trip_clean = n_h_trip_clean + 1; end
            if trip_n, n_h_trip_noisy = n_h_trip_noisy + 1; end
            min_margin_H_clean = min(min_margin_H_clean, margin_c);
            min_margin_H_noisy = min(min_margin_H_noisy, margin_n);
        else
            margin_c = i_diff_c - thr_c;
            margin_n = i_diff_n - thr_n;
            if trip_c, n_f_trip_clean = n_f_trip_clean + 1; end
            if trip_n, n_f_trip_noisy = n_f_trip_noisy + 1; end
            min_margin_F_clean = min(min_margin_F_clean, margin_c);
            min_margin_F_noisy = min(min_margin_F_noisy, margin_n);
        end

        % Write log row
        fprintf(fid_log,
'%s,%s,%d,%.4f,%s,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f,%d,%d\n', ...
            fname, group, wi, ws/FS, zone, ...
            i_pri_rms_c, i_sec_rms_c, ...
            i_diff_c, i_diff_n, ...
            i_bias_c, i_bias_n, ...
            thr_c, noise_budget, ...
            margin_c, margin_n, ...
            trip_c, trip_n);
    end

    % ------------------------------------------------------------------
    % 7f.  Score
    % ------------------------------------------------------------------
    fp_clean  = safe_pct(n_h_trip_clean, n_h_win);
    fp_noisy  = safe_pct(n_h_trip_noisy, n_h_win);
    det_clean = safe_pct(n_f_trip_clean, n_f_win);
    det_noisy = safe_pct(n_f_trip_noisy, n_f_win);
```

```
    pass_clean = (n_h_trip_clean == 0) && (n_f_trip_clean == n_f_win) && (n_f_win
> 0);
    pass_noisy = (n_h_trip_noisy == 0) && (n_f_trip_noisy == n_f_win) && (n_f_win
> 0);

    % Guard Inf (no windows in zone)
    if isinf(min_margin_H_clean), min_margin_H_clean = NaN; end
    if isinf(min_margin_H_noisy), min_margin_H_noisy = NaN; end
    if isinf(min_margin_F_clean), min_margin_F_clean = NaN; end
    if isinf(min_margin_F_noisy), min_margin_F_noisy = NaN; end

    status_str = '';
    if pass_clean && pass_noisy
        status_str = 'PASS/PASS';
    elseif pass_clean && ~pass_noisy
        status_str = 'PASS/FAIL ← noise exposed';
    elseif ~pass_clean
        status_str = 'FAIL/FAIL';
    end

    fprintf('  H=%d FP:%.0f%%/%.0f%% | F=%d Det:%.0f%%/%.0f%% | Margin_H:%.3f/%.3f
| Margin_F:%.3f/%.3f | %s\n', ...
        n_h_win, fp_clean, fp_noisy, ...
        n_f_win, det_clean, det_noisy, ...
        nanval(min_margin_H_clean), nanval(min_margin_H_noisy), ...
        nanval(min_margin_F_clean), nanval(min_margin_F_noisy), ...
        status_str);

    % Write results row
    fprintf(fid_res,
'%s,%s,%d,%.4f,%d,%.2f,%.2f,%d,%.2f,%.2f,%.4f,%.4f,%.4f,%.4f,%s,%s\n', ...
        fname, group, trans_row, ER, ...
        n_h_win, fp_clean, fp_noisy, ...
        n_f_win, det_clean, det_noisy, ...
        nanval(min_margin_H_clean), nanval(min_margin_H_noisy), ...
        nanval(min_margin_F_clean), nanval(min_margin_F_noisy), ...
        ternary(pass_clean,'PASS','FAIL'), ternary(pass_noisy,'PASS','FAIL'));

    % Save for summary
    summary_idx = summary_idx + 1;
    summary(summary_idx).fname         = fname;
    summary(summary_idx).group         = group;
    summary(summary_idx).pass_clean    = pass_clean;
    summary(summary_idx).pass_noisy    = pass_noisy;
    summary(summary_idx).margin_H_n    = nanval(min_margin_H_noisy);
    summary(summary_idx).margin_F_n    = nanval(min_margin_F_noisy);
    summary(summary_idx).margin_H_c    = nanval(min_margin_H_clean);
    summary(summary_idx).margin_F_c    = nanval(min_margin_F_clean);
end

fclose(fid_log);
fclose(fid_res);

% ========================================================================
% 8.  CONSOLE SUMMARY
% ========================================================================
fprintf('\n');
fprintf('╔═══════════════════════════════════════════════════════════════╗\n');
fprintf('║                WORST-CASE NOISE SIMULATION SUMMARY            ║\n');
```

```
fprintf('╟──────────────────────────────────────────────────────────────────────╢\n');

n_total      = summary_idx;
n_pass_clean = sum([summary.pass_clean]);
n_pass_noisy = sum([summary.pass_noisy]);
n_exposed    = sum([summary.pass_clean] & ~[summary.pass_noisy]);

fprintf('│  Files processed     : %3d                                    │\n',
n_total);
fprintf('│  PASS (clean)        : %3d / %d                               │\n',
n_pass_clean, n_total);
fprintf('│  PASS (noisy)        : %3d / %d                               │\n',
n_pass_noisy, n_total);
fprintf('│  Noise exposed FAIL  : %3d  (pass clean → fail noisy)         │\n',
n_exposed);
fprintf('╟──────────────────────────────────────────────────────────────────────╢\n');

% Worst Healthy margin (noisy) — smallest positive value = most at risk of FP
all_mH = [summary.margin_H_n];
[mH_min, mH_idx] = min(all_mH);
fprintf('│  Worst Healthy margin (noisy)                                    │\n');
fprintf('│    %.4f A  →  %s / %s\n', mH_min, summary(mH_idx).group,
summary(mH_idx).fname);

% Worst Fault margin (noisy) — smallest value = weakest detection
all_mF = [summary.margin_F_n];
[mF_min, mF_idx] = min(all_mF);
fprintf('│  Worst Fault margin (noisy)                                      │\n');
fprintf('│    %.4f A  →  %s / %s\n', mF_min, summary(mF_idx).group,
summary(mF_idx).fname);

fprintf('╟──────────────────────────────────────────────────────────────────────╢\n');
fprintf('│  NOISE BUDGET at worst operating points:                         │\n');
fprintf('│    ADC offset  : %.4f A per channel (×2 = %.4f A)                │\n',
ADC_OFFSET_A, 2*ADC_OFFSET_A);
fprintf('│    CT error    : %.4f frac → at I_bias=3.2A: %.4f A              │\n',
CT_ERROR_FRAC, 2*CT_ERROR_FRAC*3.2);
fprintf('│    Total worst : 0.016 + 0.026 × I_bias  (README Sec.5)          │\n');
fprintf('╟──────────────────────────────────────────────────────────────────────╢\n');

if n_exposed > 0
    fprintf('│  ⚠  FILES WHERE NOISE CAUSES FAILURE:                          │\n');
    for k = 1:n_total
        if summary(k).pass_clean && ~summary(k).pass_noisy
            fprintf('│    %s / %s\n', summary(k).group, summary(k).fname);
        end
    end
else
    fprintf('│  ✓  No files flipped by worst-case noise — margins adequate
│\n');
end
fprintf('└──────────────────────────────────────────────────────────────────────┘\n');
fprintf('\nDone. Results in %s/\n', out_dir);

% ========================================================================
% LOCAL FUNCTIONS
% ========================================================================
```

```
function row = findTransition(I1, win)
N = length(I1);
best_row = win + 1;  best_ratio = 0;
for i = win+1 : N-win
    rb = sqrt(mean(I1(i-win : i-1).^2));
    ra = sqrt(mean(I1(i     : i+win-1).^2));
    r  = (rb > 0) * ra / max(rb, 1e-9);
    if r > best_ratio, best_ratio = r; best_row = i; end
end
row = best_row;
end

function trip = tripLogic(id, ib, mode, s1, pu, knee, s2, off2, ss2, pus2)
if mode == 1
    thr = (ib < knee) * (s1*ib + pu) + (ib >= knee) * (s2*ib - off2);
    trip = id > thr;
else
    trip = id > ss2*ib + pus2;
end
end

function thr = computeThreshold(ib, mode, s1, pu, knee, s2, off2, ss2, pus2)
if mode == 1
    if ib < knee, thr = s1*ib + pu; else, thr = s2*ib - off2; end
else
    thr = ss2*ib + pus2;
end
end

function v = safe_pct(num, den)
if den == 0, v = NaN; else, v = 100*num/den; end
end

function v = nanval(x)
if isnan(x), v = -999; else, v = x; end
end

function s = ternary(cond, a, b)
if cond, s = a; else, s = b; end
end
```

Firmware STM32_Differential_Relay_V9.ino

```
/* ************************************************** */
/*                                                    */
/*                                                    */
/*                                                    */
/*                                                    */
/*                                                    */
/*                                                    */
/*                                                    */
/*                                                    */
/*                                                    */
/* ************************************************** */

// ==========================================
// differential_relay_v9.ino
// CHANGE LOG v8 → v9:
//   [1] ลบ TURN_RATIO, DELTA_FACTOR, GAIN_PRI, GAIN_SEC_AUTO,
//       GAIN_SEC_HARDCODE ทั้งหมดออก
//   [2] เพิ่ม EFFECTIVE_RATIO (float, runtime) — ตัวแปรตัวเดียว
//       แทนระบบ 3 ตัวเดิม คำนวณจาก I_pri_avg / I_sec_avg ตอน Boot
//   [3] ลบ runTurnRatioCalibration() + runGainCalibration() ออก
//       เหลือ runEffectiveRatioCalibration() ฟังก์ชันเดียว
//   [4] measureRMSBlocking() เรียบง่ายขึ้น ไม่มี gain_sec parameter
//   [5] Main loop: i_pri_comp = I_pri / EFFECTIVE_RATIO
//                  i_diff = |i_pri_comp - I_sec|
//   ผลลัพธ์: ไม่ต้อง detect Yy/Yd/Dy ทำงานถูกต้องกับทุก configuration
// ==========================================

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SH110X.h>

// ==========================================
// การตั้งค่าหน้าจอ OLED 1.3" (SH1106)
// ==========================================
#define i2c_Address 0x3c
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SH1106G display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// ==========================================
// กำหนดขาพิน (Pins Definition)
// ==========================================
const int pin_ADC[6] = {PA0, PA1, PA2, PA3, PA4, PA5};
// [0]=Pri_A, [1]=Sec_A, [2]=Pri_B, [3]=Sec_B, [4]=Pri_C, [5]=Sec_C

#define BTN_PIN   PB12
```

```
#define BUZZER_PIN PB0

#define LED_A     PB1
#define LED_B     PB3
#define LED_C     PB4

// ==========================================
// CT Parameters
// CT_RATIO = (I_pri_rated / I_sec_rated) / R_burden  [A/V]
// MSQ-0.66: 30A/5A, R_burden=0.10Ω → CT_RATIO = 60.0 A/V
// ==========================================
const float R_BURDEN     = 0.10f;
const float CT_RATIO_PRI = (30.0f / 5.0f) / R_BURDEN;   // 60.0 A/V
const float CT_RATIO_SEC = (30.0f / 5.0f) / R_BURDEN;   // 60.0 A/V

// ==========================================
// Hardcode Gain Correction
//
// ใช้ชดเชย error ของ CT แต่ละตัว เพื่อให้ค่าที่อ่านได้ตรงกับ Clamp Meter
// วิธีหาค่า: GAIN = I_clamp / I_stm (วัดที่กระแส 60-80% ของพิกัด)
//
// เลือก MODE ด้วย #define ด้านล่าง:
//   GAIN_MODE 1 — Pri รวม + Sec รวม (2 ค่า)
//   GAIN_MODE 2 — แยกรายตัว 6 ช่อง (PA, SA, PB, SB, PC, SC)
// ==========================================
#define GAIN_MODE  2   // <-- เปลี่ยนเป็น 2 เพื่อใช้ gain แยกรายตัว

#if GAIN_MODE == 1
  // Mode 1: Pri ทั้ง 3 เฟสใช้ค่าเดียว / Sec ทั้ง 3 เฟสใช้ค่าเดียว
  // แก้ค่าหลังวัด CT ratio แต่ละชุดแล้ว
  const float GAIN_PRI_ALL = 1.000f;   // A,B,C Primary รวม
  const float GAIN_SEC_ALL = 1.000f;   // A,B,C Secondary รวม

#elif GAIN_MODE == 2
  // Mode 2: แยกรายตัว — ลำดับตรงกับ pin_ADC[]
  // [0]=Pri_A  [1]=Sec_A  [2]=Pri_B  [3]=Sec_B  [4]=Pri_C  [5]=Sec_C
  const float GAIN_CH[6] = {
    1.104f,   // [0] Pri_A
    1.100f,   // [1] Sec_A
    1.146f,   // [2] Pri_B
    1.137f,   // [3] Sec_B
    1.100f,   // [4] Pri_C
    1.141f,   // [5] Sec_C
  };
#else
  #error "GAIN_MODE must be 1 or 2"
#endif
```

```
// Helper inline — คืน gain ของช่อง i ตาม GAIN_MODE ที่เลือก
inline float getGain(int i) {
#if GAIN_MODE == 1
  return (i % 2 == 0) ? GAIN_PRI_ALL : GAIN_SEC_ALL;
#else
  return GAIN_CH[i];
#endif
}

// ==========================================
// EFFECTIVE_RATIO (runtime)
//
// = I_pri_avg / I_sec_avg ที่วัดได้ตอน Boot ขณะ Healthy
// รวม TURN_RATIO + DELTA_FACTOR + GAIN mismatch ทุกอย่างในตัวเดียว
//
// ใช้ใน loop: i_pri_comp = I_pri / EFFECTIVE_RATIO
//             i_diff = |i_pri_comp - I_sec|  ≈ 0 ขณะ Healthy
//
// Fallback = 1.0f ถ้า Cal ล้มเหลว (กระแสน้อยเกิน)
// ==========================================
float EFFECTIVE_RATIO = 1.0f;   // runtime — set ใน
runEffectiveRatioCalibration()

// ==========================================
// Protection Mode
//   1 = Dual Slope (IEC): ch1→ 0.10*Ib+0.45 | ch2→ 0.50*Ib+1.55
//   2 = Single Slope:     0.60*Ib + 10.0
// ==========================================
#define PROTECTION_MODE  1

const float SLOPE1     = 0.10f;
const float PICKUP     = 0.45f;
const float KNEE_POINT = 5.0f;
const float SLOPE2     = 0.50f;
const float OFFSET2    = 1.55f;

const float SLOPE_S2   = 0.60f;
const float PICKUP_S2  = 10.0f;

// ==========================================
// Calibration Constants
// ==========================================
#define CALIB_SAMPLES          2000   // DC offset samples
#define CALIB_COUNTDOWN           3   // วินาที countdown DC offset
#define TR_COUNTDOWN             10   // วินาที countdown TR cal
#define TR_MEASURE_MS         10000   // ระยะเวลาวัด TR (ms)
#define TR_SAMPLE_INTERVAL_US   500   // sampling interval (us)
#define TR_MIN_CURRENT          0.5f  // A — กระแสขั้นต่ำ
```

```
#define DC_WARN_TOLERANCE       0.10f // V — warning threshold

// Safety clamp สำหรับ GAIN_SEC_AUTO
// ถ้าคำนวณออกมาแล้วอยู่นอกช่วงนี้ → น่าจะผิดพลาด → fallback

float dc_offset[6];

// ==========================================
// Non-blocking Timers
// ==========================================
unsigned long prevSampleMicros = 0;
const unsigned long SAMPLE_INTERVAL_US = 1000;

unsigned long prevRMSMillis = 0;
const unsigned long RMS_INTERVAL_MS = 40;

unsigned long prevOLEDMillis = 0;
const unsigned long OLED_INTERVAL_MS = 250;

// ==========================================
// RMS Accumulators
// ==========================================
double sum_sq[6]    = {0,0,0,0,0,0};
int    sample_count = 0;

// ==========================================
// Results
// ==========================================
float rms_current[6] = {0,0,0,0,0,0};
float i_diff[3]      = {0,0,0};
float i_bias[3]      = {0,0,0};
bool  is_fault[3]    = {false,false,false};

// ==========================================
// Button & Display
// ==========================================
int  current_page   = 0;
bool last_btn_state = HIGH;
unsigned long last_btn_millis = 0;
const unsigned long DEBOUNCE_MS = 50;

// ==========================================
// shouldTrip()
// ==========================================
bool shouldTrip(float id, float ib) {
#if PROTECTION_MODE == 1
  float threshold = (ib < KNEE_POINT)
                  ? (SLOPE1 * ib + PICKUP)
```

```
                    : (SLOPE2 * ib - OFFSET2);
  return (id > threshold);
#elif PROTECTION_MODE == 2
  return (id > SLOPE_S2 * ib + PICKUP_S2);
#else
  #error "PROTECTION_MODE must be 1 or 2"
#endif
}

// ==========================================
// measureRMSBlocking()
//
// ออกมาเป็น A หลัง apply CT_RATIO + gain แล้ว
// วัด RMS แบบ blocking — ใช้เฉพาะตอน calibration
// ไม่มี gain คูณ เพราะ EFFECTIVE_RATIO จัดการทุกอย่างใน loop จริง
// ออกมาเป็น A หลัง apply CT_RATIO × Gain แล้ว
// ==========================================

void measureRMSBlocking(float* out_rms, int nSamples, int intervalUs) {
  double acc[6] = {0,0,0,0,0,0};
  for (int s = 0; s < nSamples; s++) {
    for (int i = 0; i < 6; i++) {
      int   raw  = analogRead(pin_ADC[i]);
      float v_ac = ((raw / 4095.0f) * 3.3f) - dc_offset[i];
      acc[i]    += (double)(v_ac * v_ac);
    }
    delayMicroseconds(intervalUs);
  }
  for (int i = 0; i < 6; i++) {
    float v_rms = sqrt((float)(acc[i] / nSamples));
    float ratio  = (i % 2 == 0) ? CT_RATIO_PRI : CT_RATIO_SEC;
    out_rms[i]  = v_rms * ratio * getGain(i);
  }
}

// ==========================================
// STEP 1: DC Offset Calibration
// ==========================================
void runDCOffsetCalibration() {
  for (int sec = CALIB_COUNTDOWN; sec >= 1; sec--) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SH110X_WHITE);
    display.setCursor(5, 0);  display.println("== DC OFFSET CAL ==");
    display.setCursor(5, 14); display.println("No current required");
    display.setCursor(5, 24); display.println("CT can stay clamped");
    display.setCursor(5, 48); display.print("Starting in: ");
    display.print(sec); display.print("s");
```

```
    display.display();
    delay(1000);
  }

  display.clearDisplay();
  display.setCursor(5, 0);  display.println("== DC OFFSET CAL ==");
  display.setCursor(5, 18); display.println("Measuring...");
  display.display();

  double acc[6] = {0,0,0,0,0,0};
  for (int s = 0; s < CALIB_SAMPLES; s++) {
    for (int i = 0; i < 6; i++) acc[i] += analogRead(pin_ADC[i]);
    delayMicroseconds(500);
  }
  for (int i = 0; i < 6; i++) {
    dc_offset[i] = ((float)(acc[i] / CALIB_SAMPLES) / 4095.0f) * 3.3f;
  }

  display.clearDisplay();
  display.setCursor(0, 0); display.println("Offset(V)  status");
  display.drawFastHLine(0, 9, 128, SH110X_WHITE);

  const char* labels[6] = {"PA","SA","PB","SB","PC","SC"};
  for (int i = 0; i < 6; i++) {
    int y = 12 + i * 9;
    display.setCursor(0, y);
    display.print(labels[i]); display.print(":"); display.print(dc_offset[i],
3);
    display.setCursor(90, y);
    display.print(fabsf(dc_offset[i] - 1.65f) <= DC_WARN_TOLERANCE ? "[OK]" :
"[!!]");
  }
  display.display();
  delay(5000);

  Serial.println("=== DC Offset Calibration ===");
  const char* sl[6] = {"Pri_A","Sec_A","Pri_B","Sec_B","Pri_C","Sec_C"};
  for (int i = 0; i < 6; i++) {
    Serial.print(sl[i]); Serial.print(" = "); Serial.print(dc_offset[i], 4);
    Serial.println(fabsf(dc_offset[i]-1.65f) <= DC_WARN_TOLERANCE ? "V [OK]" :
"V [!!]");
  }
}

// ==========================================
// STEP 2: Effective Ratio Calibration  [v8]
//
// แทน TURN_RATIO + DELTA_FACTOR + GAIN_SEC_AUTO ทั้งหมดด้วย
```

```
// EFFECTIVE_RATIO = I_pri_avg / I_sec_avg ที่วัดขณะ Healthy
//
// ใน loop: i_pri_comp = I_pri / EFFECTIVE_RATIO ≈ I_sec ขณะ Healthy
//          i_diff = |i_pri_comp - I_sec| ≈ 0 ขณะ Healthy
//                                          พุ่งสูงขณะ Fault
//
// Fallback: EFFECTIVE_RATIO = 1.0f ถ้ากระแสน้อยเกิน TR_MIN_CURRENT
// ==========================================
void runEffectiveRatioCalibration() {
  // Countdown — user เปิดโหลดระหว่างนี้ได้เลย
  for (int sec = TR_COUNTDOWN; sec >= 1; sec--) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SH110X_WHITE);
    display.setCursor(5, 0);  display.println("== RATIO CAL (v8) ==");
    display.setCursor(5, 14); display.println("Apply normal load NOW");
    display.setCursor(5, 24); display.println("Need I > 0.5A");
    display.setCursor(5, 36); display.println("Keep load stable");
    display.setCursor(5, 50); display.print("Measuring in: ");
    display.print(sec); display.print("s");
    display.display();
    delay(1000);
  }

  display.clearDisplay();
  display.setCursor(5, 0);  display.println("== RATIO CAL (v8) ==");
  display.setCursor(5, 18); display.println("Measuring 10s...");
  display.setCursor(5, 30); display.println("Keep load stable!");
  display.display();

  const int trSamples = (TR_MEASURE_MS * 1000) / TR_SAMPLE_INTERVAL_US;
  float measured[6]   = {0,0,0,0,0,0};
  measureRMSBlocking(measured, trSamples, TR_SAMPLE_INTERVAL_US);

  // เฉลี่ยข้าม 3 เฟสเพื่อลด noise จากโหลดไม่สมดุล
  float i_pri_avg = (measured[0] + measured[2] + measured[4]) / 3.0f;
  float i_sec_avg = (measured[1] + measured[3] + measured[5]) / 3.0f;

  Serial.println("=== Effective Ratio Calibration ===");
  Serial.print("I_pri avg = "); Serial.print(i_pri_avg, 3); Serial.println("
A");
  Serial.print("I_sec avg = "); Serial.print(i_sec_avg, 3); Serial.println("
A");

  // ตรวจสอบกระแสขั้นต่ำ
  if (i_pri_avg < TR_MIN_CURRENT || i_sec_avg < TR_MIN_CURRENT) {
    display.clearDisplay();
    display.setCursor(5, 0);  display.println("== RATIO CAL (v8) ==");
```

```
    display.setCursor(5, 14); display.println("FAILED! Current<0.5A");
    display.setCursor(5, 26); display.print("Ipri="); display.print(i_pri_avg,
2);
    display.print(" Isec="); display.print(i_sec_avg, 2);
    display.setCursor(5, 40); display.println("Using ER=1.0 (fallback)");
    display.setCursor(5, 52); display.println("Restart to retry");
    display.display();

    EFFECTIVE_RATIO = 1.0f;
    Serial.println("[!!] FAILED — EFFECTIVE_RATIO=1.0 (fallback), please
restart");
    delay(8000);
    return;
  }

  // คำนวณ EFFECTIVE_RATIO
  EFFECTIVE_RATIO = i_pri_avg / i_sec_avg;

  Serial.print("EFFECTIVE_RATIO = "); Serial.println(EFFECTIVE_RATIO, 4);
  Serial.println("[OK] Applied");

  display.clearDisplay();
  display.setCursor(5, 0);  display.println("== RATIO CAL (v8) ==");
  display.setCursor(5, 14); display.print("Ipri="); display.print(i_pri_avg,
3); display.print("A");
  display.setCursor(5, 26); display.print("Isec="); display.print(i_sec_avg,
3); display.print("A");
  display.setCursor(5, 40); display.print("ER = ");
display.print(EFFECTIVE_RATIO, 4);
  display.setCursor(5, 52); display.println("[OK] Applied");
  display.display();
  delay(4000);
}

// ==========================================
// SETUP
// ==========================================
void setup() {
  Serial.begin(500000);

  analogReadResolution(12);
  for (int i = 0; i < 6; i++) pinMode(pin_ADC[i], INPUT_ANALOG);

  pinMode(BTN_PIN,   INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT); digitalWrite(BUZZER_PIN, HIGH);
  pinMode(LED_A, OUTPUT); digitalWrite(LED_A, LOW);
  pinMode(LED_B, OUTPUT); digitalWrite(LED_B, LOW);
  pinMode(LED_C, OUTPUT); digitalWrite(LED_C, LOW);
```

```
  Wire.begin();
  Wire.setClock(400000);
  delay(250);

  if (!display.begin(i2c_Address, true)) {
    Serial.println("OLED Init Failed — all fallback values");
    for (int i = 0; i < 6; i++) dc_offset[i] = 1.65f;
    EFFECTIVE_RATIO = 1.0f;
  } else {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SH110X_WHITE);
    display.setCursor(10, 20); display.print("Relay Booting v9...");
    display.display();
    delay(500);

    runDCOffsetCalibration();          // Step 1: ไม่มีโหลด — วัด DC offset
    runEffectiveRatioCalibration();    // Step 2: มีโหลด  — วัด EFFECTIVE_RATIO
  }

  Serial.println("=== System Ready ===");
  Serial.print("EFFECTIVE_RATIO="); Serial.println(EFFECTIVE_RATIO, 4);
  Serial.println("RMS_Pri_A,RMS_Sec_A,RMS_Pri_B,RMS_Sec_B,RMS_Pri_C,RMS_Sec_C,
"
                 "Idiff_A,Ibias_A,Fault_A,Idiff_B,Ibias_B,Fault_B,Idiff_C,Ibia
s_C,Fault_C");
}

// ==========================================
// LOOP
// ==========================================
void loop() {
  unsigned long currentMicros = micros();
  unsigned long currentMillis = millis();

  // ====================================================
  // 1. Sampling — ทุก 1 ms
  // ====================================================
  if (currentMicros - prevSampleMicros >= SAMPLE_INTERVAL_US) {
    prevSampleMicros += SAMPLE_INTERVAL_US;
    for (int i = 0; i < 6; i++) {
      int   raw  = analogRead(pin_ADC[i]);
      float v_ac = ((raw / 4095.0f) * 3.3f) - dc_offset[i];
      sum_sq[i] += (double)(v_ac * v_ac);
    }
    sample_count++;
  }
```

```
// ====================================================
// 2. RMS + Protection — ทุก 40 ms
// ====================================================
if (currentMillis - prevRMSMillis >= RMS_INTERVAL_MS) {
  prevRMSMillis += RMS_INTERVAL_MS;

  if (sample_count > 0) {
    bool any_fault = false;

    for (int i = 0; i < 6; i++) {
      float v_rms = sqrt((float)(sum_sq[i] / sample_count));
      sum_sq[i]   = 0;
      float ratio = (i % 2 == 0) ? CT_RATIO_PRI : CT_RATIO_SEC;
      rms_current[i] = v_rms * ratio * getGain(i);
    }
    sample_count = 0;

    for (int phase = 0; phase < 3; phase++) {
      int p_idx = phase * 2;
      int s_idx = p_idx + 1;

      // [v8] EFFECTIVE_RATIO รวม TURN_RATIO + DELTA_FACTOR + GAIN mismatch
      float i_pri_comp = rms_current[p_idx] / EFFECTIVE_RATIO;
      float i_sec_comp = rms_current[s_idx];

      i_diff[phase] = fabsf(i_pri_comp - i_sec_comp);
      i_bias[phase] = (i_pri_comp + i_sec_comp) * 0.5f;

      is_fault[phase] = shouldTrip(i_diff[phase], i_bias[phase]);
      if (is_fault[phase]) any_fault = true;
    }

    digitalWrite(BUZZER_PIN, any_fault ? LOW : HIGH);
    digitalWrite(LED_A, is_fault[0] ? HIGH : LOW);
    digitalWrite(LED_B, is_fault[1] ? HIGH : LOW);
    digitalWrite(LED_C, is_fault[2] ? HIGH : LOW);

    for (int i = 0; i < 6; i++) {
      Serial.print(rms_current[i], 3); Serial.print(",");
    }
    for (int ph = 0; ph < 3; ph++) {
      Serial.print(i_diff[ph], 3); Serial.print(",");
      Serial.print(i_bias[ph], 3); Serial.print(",");
      Serial.print(is_fault[ph] ? "1" : "0");
      if (ph < 2) Serial.print(",");
    }
    Serial.println();
```

```
    }
  }

  // ====================================================
  // 3. ปุ่มกด — Non-blocking debounce
  // ====================================================
  bool current_btn_state = digitalRead(BTN_PIN);
  if (current_btn_state == LOW && last_btn_state == HIGH) {
    if (currentMillis - last_btn_millis >= DEBOUNCE_MS) {
      last_btn_millis = currentMillis;
      current_page++;
      if (current_page > 2) current_page = 0;
    }
  }
  last_btn_state = current_btn_state;

  // ====================================================
  // 4. OLED — ทุก 250 ms
  // ====================================================
  if (currentMillis - prevOLEDMillis >= OLED_INTERVAL_MS) {
    prevOLEDMillis += OLED_INTERVAL_MS;

    display.clearDisplay();
    display.drawRect(0, 0, 128, 64, SH110X_WHITE);

    display.setTextSize(1);
    display.setCursor(4, 4);
    display.print("PH-");
    const char phaseNames[] = {'A','B','C'};
    display.print(phaseNames[current_page]);
    display.setCursor(30, 4);  display.print("ER:");
    display.print(EFFECTIVE_RATIO, 2);
    display.setCursor(100, 4);
#if PROTECTION_MODE == 1
    display.print("M1");
#else
    display.print("M2");
#endif
    display.drawFastHLine(0, 14, 128, SH110X_WHITE);

    int p_idx = current_page * 2;
    int s_idx = p_idx + 1;

    display.setCursor(5, 18);
    display.print("P:"); display.print(rms_current[p_idx], 2);
display.print("A");
    display.setCursor(67, 18);
```

```
    display.print("S:"); display.print(rms_current[s_idx], 2);
display.print("A");

    display.setCursor(5, 30);
    display.print("Id:"); display.print(i_diff[current_page], 2);
    display.setCursor(67, 30);
    display.print("Ib:"); display.print(i_bias[current_page], 2);

    display.setCursor(5, 42);
    display.print("ER:"); display.print(EFFECTIVE_RATIO, 3);
    display.setCursor(67, 42);

    display.setCursor(5, 54);
    display.print("Sts: ");
    if (is_fault[current_page]) {
      display.setTextColor(SH110X_BLACK, SH110X_WHITE);
      display.print(" FAULT! ");
      display.setTextColor(SH110X_WHITE, SH110X_BLACK);
    } else {
      display.print("HEALTHY");
    }

    display.display();
  }
}
```